```c
Candidato* leggi_candidati(FILE* f, int* dim) {
    int conto = 0;
    int tmp = 0;
    //Prima passata di conto
    // Uso l'asterisco %* per scartare i dati che non mi servono ora (ne tengo uno per il corretto funzionamento di fscanf)
        while (fscanf(f, " %*s %*s %d", &tmp) == 1) {
            conto++;
        }
    if (conto == 0) {
        return NULL;
    }
    //Allocazione array
    Candidato* array = (Candidato*)malloc(conto * sizeof(Candidato));
    if (array == NULL) {
        fprintf(stderr, "Errore di allocazione\n");
        return NULL;
    }
    //Seconda passata di assegnamento
    rewind(f);
    for (int i = 0; i < conto; i++) {
        readArrayRecord(f, &array[i]);
    }
    //Assegnamento dimensione array
    *dim = conto;
    return array;
}

//Es 2
Candidato* trova_candidati(char* p_id, list p, Candidato* c, int dim_c, int* dim) {
    int i = 0;
    int conto = 0;
    //Prima passata di conto
    //Individuiamo la posizione cercata
    list tmp = p;
    Posizione posCercata = { 0 };
    while (empty(tmp) == FALSE) {
        if (strcmp(head(tmp).ID, p_id) == 0) {
            //Trovata!
            posCercata = head(tmp);
            while (i < dim_c) {
                if (c[i].settore == posCercata.settore && c[i].anniDiEsp >= posCercata.minAnniEsp) {
                    conto++;
                }
                i++;
            }
        }
        tmp = tail(tmp);
    }
    //Se non si trova la posizione
    if (strcmp(posCercata.ID,"") == 0) {
        printf("\nPosizione non presente!\n");
        return NULL;
    }
    //Allocazione array
    Candidato* candSelezionati = (Candidato*)malloc(sizeof(Candidato) * conto);
    if (candSelezionati == NULL) {
        fprintf(stderr, "Errore di allocazione\n");
        return NULL;
    }
    //Seconda passata
```

Fondamenti

Il linguaggio

In informatica, il linguaggio è inteso come sistema matematico, che consenta di rispondere a domande come:

> - Quali sono le frasi lecite?
> - Si può stabilire se una frase appartiene al linguaggio?
> - Come si stabilisce il significato di una frase?
> - Quali elementi linguistici primitivi?

Dato un algoritmo, un programma è la sua descrizione in un particolare **linguaggio di programmazione**:

> Un linguaggio di programmazione è una notazione formale che può essere usata per descrivere algoritmi. Due aspetti fondamentali del linguaggio sono sintassi e semantica.

Definizione:

- Un **linguaggio** è un insieme di **frasi**
- Una **frase** è una **sequenza di simboli** appartenenti a un certo alfabeto

Proprietà desiderabili:

- Un linguaggio deve essere **effettivamente generabile**
- Un linguaggio di programmazione deve essere **decidibile**

Altri elementi:

- **Alfabeto V** (o vocabolario o lessico) è **l'insieme dei simboli** con cui si costruiscono le frasi.
- **Universo linguistico V*** di un alfabeto V è l'insieme di tutte le frasi (sequenze finite di lunghezza arbitraria) di elementi V.
- **Linguaggio L** su un alfabeto V è un sottoinsieme di V*

Ma come specificare il sottoinsieme di V* che definisce il linguaggio?

- Specificando in modo **formale e preciso la sintassi** delle frasi del linguaggio, tramite:
  - Una grammatica formale
  - Una **notazione matematica** che consenta di esprimere in modo rigoroso la sintassi di un linguaggio

Grammatica formale

Costituisce una **quadrupla** <**VT**, **VN**, **P**, **S**> dove:

> **VT** è un insieme finito di **simboli terminali**.
> **VN** è un insieme finito di **simboli non terminali**.
> **P** è un insieme finito di produzioni, ossia di **regole di riscrittura**.
> **S** è un particolare simbolo non-terminale detto **simbolo iniziale** o **scopo della grammatica**.

BNF:

Meta-sintassi attraverso cui è possibile descrivere la sintassi dei linguaggi di programmazione.

Nelle **grammatiche BNF** le produzioni hanno forma:

$$x ::= A$$

- $x \in VN$ è un simbolo non terminale
- A è una sequenza di simboli ciascuno appartenente all'alfabeto $VN \cup VT$

Una grammatica BNF dunque definisce un **linguaggio sull'alfabeto terminale VT** mediante un **meccanismo di derivazione** (o riscrittura).

Data dunque una **grammatica** G:

> Si dice Linguaggio L_G generato da G l'insieme delle frasi di V derivabili dal simbolo iniziale S applicando le produzioni P.

Le **frasi** di un linguaggio di programmazione vengono dette **programmi** di tale linguaggio.

Derivazione "Left most":

A partire dallo scopo della grammatica, si riscrive sempre il simbolo non-terminale più a sinistra.

Compilato

Cos'è:

Un linguaggio compilato è un linguaggio di programmazione in cui il codice sorgente viene **tradotto in anticipo** da un **compilatore** in un **file eseguibile** (linguaggio macchina).
Questo file può essere eseguito direttamente dal sistema operativo senza bisogno di interprete. Ad es: C

Pro:

- **Prestazioni elevate**: il programma gira molto più velocemente, perché è già in linguaggio macchina.
- **Eseguibile indipendente**: una volta compilato, il file può essere distribuito ed eseguito senza bisogno del compilatore.
- **Ottimizzazioni**: il compilatore può ottimizzare il codice, riducendo consumi di memoria e aumentando l'efficienza.
- **Maggiore controllo**: spesso i linguaggi compilati permettono un controllo più diretto sull'hardware e sulle risorse.

Contro:

- **Tempi di compilazione**: prima di eseguire il programma bisogna attendere la compilazione (che può essere lunga per progetti grandi).
- **Debug meno immediato**: gli errori si scoprono solo dopo la compilazione, e a volte i messaggi sono più criptici.
- **Portabilità limitata**: un eseguibile compilato per un sistema operativo/architettura spesso non funziona su un altro (es. programma compilato su Windows non gira su Linux senza ricompilazione). Il target è sia il **processore** che il **sistema operativo**

Interpretato

Cos'è:

Un linguaggio interpretato è un linguaggio il cui codice viene eseguito direttamente da un programma chiamato interprete, riga per riga. Es: BASIC.

Pro:

- **Portabilità**: basta avere l'interprete installato, non serve ricompilare il programma per ogni sistema operativo.
- **Sviluppo rapido**: puoi eseguire subito il codice e testare modifiche senza compilazione.
- **Debug più semplice**: errori vengono segnalati subito all'esecuzione, spesso con messaggi dettagliati.

Contro:

- **Prestazioni più basse**: l'esecuzione è più lenta, perché il codice viene tradotto "al volo" ogni volta.
- **Dipendenza dall'interprete**: per eseguire il programma serve avere installato l'interprete giusto.
- **Distribuzione meno immediata**: non hai un unico file eseguibile indipendente, ma devi distribuire anche l'interprete (o il codice sorgente).

La macchina intermedia

Definizione:

È una via di mezzo tra un linguaggio compilato e uno interpretato.

Come funziona:

- Il codice sorgente non viene tradotto direttamente in linguaggio macchina, ma nemmeno interpretato riga per riga.
- Viene tradotto in un bytecode intermedio "più vicino alla macchina" ma ancora indipendente dall'hardware.
- Questo bytecode viene poi eseguito da una macchina virtuale (VM) che lo interpreta e lo compila "al volo".
- Esempi: Python, Java.
-

ASCII

Cos'è:

ASCII è l'acronimo di **American Standard Code for Information Interchange**, uno standard di codifica dei caratteri sviluppato negli anni '60 che assegna un **numero binario** (o decimale) a lettere, numeri, simboli di punteggiatura e caratteri di controllo.

Cenni storici:

Nasce nel 1968, con l'obiettivo di associare a ciascun carattere un numero intero.

Inizialmente erano codici a 7 bit, poi 8.

Tabella ASCII:

Byte	Cod.	Char
00000000	0	Null
00000001	1	Start of heading
00000010	2	Start of text
00000011	3	End of text
00000100	4	End of transmit
00000101	5	Enquiry
00000110	6	Acknowledge
00000111	7	Audible bell
00001000	8	Backspace
00001001	9	Horizontal tab
00001010	10	Line feed
00001011	11	Vertical tab
00001100	12	Form Feed
00001101	13	Carriage return
00001110	14	Shift out
00001111	15	Shift in
00010000	16	Data link escape
00010001	17	Device control 1
00010010	18	Device control 2
00010011	19	Device control 3
00010100	20	Device control 4
00010101	21	Neg. acknowledge
00010110	22	Synchronous idle
00010111	23	End trans. block
00011000	24	Cancel
00011001	25	End of medium
00011010	26	Substitution
00011011	27	Escape
00011100	28	File separator
00011101	29	Group separator
00011110	30	Record Separator
00011111	31	Unit separator

Byte	Cod.	Char
00100000	32	Spc
00100001	33	!
00100010	34	"
00100011	35	#
00100100	36	$
00100101	37	%
00100110	38	&
00100111	39	'
00101000	40	(
00101001	41	)
00101010	42	*
00101011	43	+
00101100	44	,
00101101	45	-
00101110	46	.
00101111	47	/
00110000	48	0
00110001	49	1
00110010	50	2
00110011	51	3
00110100	52	4
00110101	53	5
00110110	54	6
00110111	55	7
00111000	56	8
00111001	57	9
00111010	58	:
00111011	59	;
00111100	60	<
00111101	61	=
00111110	62	>
00111111	63	?

Byte	Cod.	Char
01000000	64	@
01000001	65	A
01000010	66	B
01000011	67	C
01000100	68	D
01000101	69	E
01000110	70	F
01000111	71	G
01001000	72	H
01001001	73	I
01001010	74	J
01001011	75	K
01001100	76	L
01001101	77	M
01001110	78	N
01001111	79	O
01010000	80	P
01010001	81	Q
01010010	82	R
01010011	83	S
01010100	84	T
01010101	85	U
01010110	86	V
01010111	87	W
01011000	88	X
01011001	89	Y
01011010	90	Z
01011011	91	[
01011100	92	\
01011101	93	]
01011110	94	^
01011111	95	_

Byte	Cod.	Char
01100000	96	`
01100001	97	a
01100010	98	b
01100011	99	c
01100100	100	d
01100101	101	e
01100110	102	f
01100111	103	g
01101000	104	h
01101001	105	i
01101010	106	j
01101011	107	k
01101100	108	l
01101101	109	m
01101110	110	n
01101111	111	o
01110000	112	p
01110001	113	q
01110010	114	r
01110011	115	s
01110100	116	t
01110101	117	u
01110110	118	v
01110111	119	w
01111000	120	x
01111001	121	y
01111010	122	z
01111011	123	{
01111100	124	|
01111101	125	}
01111110	126	~
01111111	127	Del

L'escaping:

L'escaping è il processo di anteporre un **carattere speciale**, come la barra rovesciata (`\`), a un altro carattere per indicare che deve essere **trattato letteralmente** e non interpretato secondo la sua **funzione speciale**.

Link:

Tipi primitivi

Assegnamento

Definizione:

L'assegnamento è l'astrazione della modifica distruttiva del contenuto della cella di memoria denotata dalla variabile.

Sintassi:

```
<espressione> = <assegnamento>
```

Assegnamento multiplo:

Un assegnamento è considerato un'espressione con un risultato. E' quindi possibile eseguire **assegnamenti multipli** su una sola riga di codice.

Es:

```
int x,y = 1, z=0; z=x=y=2;
```

Associativo a destra!! Dunque equivale a:

```
z=(x=(y=2));
>```

Cioè:

```c
 y=2;
 x=2;
 z=2;
```

Nota:

```
z=2=y; Errato!
```
```

Bit e Byte

Cosa sono:

Il Bit è l'**unita di informazione di base** (0-1).

Il Byte (8 bit) è la più piccola quantità di memoria indirizzabile in molte architetture di calcolatori.

Quanti oggetti posso codificare con n bit?

$$2^n$$

E il numero minimo di bit sufficiente a codificare N oggetti distinti?

$$N <= 2^n \quad n = log_2 N$$

Link:
Il linguaggio

Conversioni di tipo

Esistono di due tipi:

- **Implicita** (coercion):
  - effettuata dal compilatore in presenza di espressioni eterogenee
- **Esplicita** (typecasting):
  - ottenuta dal programmatore utilizzando uno specifico operatore (cast)

Regola adottata in C:

Sono eseguibili le **espressioni eterogenee** in cui tutti i tipi referenziati risultano **compatibili** (ovvero che risultano omogenei dopo l'applicazione della regola automatica di conversione implicita di tipo del C).

Compatibilità di tipo:

Consiste nella possibilità di usare, entro certi limiti, **oggetti di un tipo** al posto di **oggetti di un altro tipo**.

Un tipo **T1** è compatibile con un tipo **T2** se il dominio D1 di **T1** è contenuto nel dominio **D2** di **T2**

Es:

- `int` è compatibile con `float` perché $Z \subset R$ – ma `float` non è compatibile con `int`

⚠ **C per convertire un numero reale in un numero intero usa il troncamento → nessun arrotondamento e perdita di informazione.**

In pratica:

In pratica il tipo di **dimensione minore** viene convertito in quello di **dimensione maggiore**.

Gerarchia di conversione delle espressioni eterogenee:

```
int < long < float < double < long double
```

⚠ **Ocio! La conversione può portare perdita di informazioni:**

```
int x;
char y;
```

```
double r;
……
x = y; /* char -> int*/
x = y+x;
r = y; /* char -> int -> double*/
x = r; /* troncamento*/
```

Il caso peggiore è:

```
float x = 0.99
int x //x per troncamento diventa 0 --> tragiche conseguenze
```

Conversione esplicita di tipo: Cast:

In qualunque espressione è possibile **forzare una particolare conversione** utilizzando l'**operatore di cast**:

```
(<tipo>) <espressione>
```

Es:

```
int i=3,k=2;
long double x=7.77;
double y=7.1;
i = (int) sqrt(384);
i = (int) x % (int)y;
y = (double) i / k; // y = 1,5
```

L'operatore `cast` ha precedenza rispetto all'operatore `/` e viene forzato prima della divisione.

Link:
Linguaggio tipizzato

Espressioni

Cosa sono:

• Il C è un linguaggio **basato su espressioni**

• Un' espressione è una notazione che **denota un valore** mediante un **processo di valutazione**

• Una espressione può essere **semplice** o **composta** (tramite aggregazione di altre espressioni)

Effetti collaterali:

• Le espressioni che contengono variabili, oltre a denotare un valore, possono a volte comportare **effetti collaterali** sulle variabili coinvolte

• Un **effetto collaterale** è una modifica del valore della variabile (R-value) causato da particolari operatori:

- Operatore di **assegnamento**
- Operatori di **incremento e decremento**

Es **espressione semplice**:

```
3
```

Es **espressione composta**:

```
3 + 5 = 8
```

Link:
C

Sintassi e semantica

Sintassi:

> L'insieme di **regole formali** per la scrittura di programmi in un linguaggio, che dettano le modalità per costruire frasi corrette nel linguaggio stesso.

Semantica:

> L'insieme dei significati da attribuire alle frasi (sintatticamente corrette) costruite nel linguaggio.

Le notazioni formali:

Le regole sintattiche sono espresse attraverso **notazioni formali**:

- **BNF** (Backus-Naur Form)
- **EBNF** (Extended BNF)
- **Diagrammi sintattici**

Esempio di sintassi di un numero naturale in **EBNF**:

```
<naturale> ::=
0 | <cifra-non-nulla>{<cifra>}
<cifra-non-nulla> ::=
1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
<cifra> ::=
0 | <cifra-non-nulla>
```

La semantica è esprimibile:

- A parole (poco precisa e ambigua)
- Mediante azioni $\rightarrow$ **semantica operazionale**
- Mediante funzioni matematica $\rightarrow$ **semantica denotazionale**
- Mediante formule logiche $\rightarrow$ **semantica assiomatica**

C

Caratteristiche:

- Questo linguaggio nacque nel 1972 da Kernigham & Ritchie
- Sequenziale, imperativo, strutturato a blocchi, basato su espressioni
- Usabile anche come linguaggio di sistema --> linguaggio principe per questo scopo (kernel di windows, mac, linux
    - Portabile, efficiente (compilato), sintetico
        - Tuttavia a volte poco leggibile
        - Ciò è lecito:
          `b++++++++++++++++;`
- Sequenziale: l'ordine delle righe è fondamentale
- Strutturato a blocchi come il Pascal

Linguaggio tipizzato

Cosa intendiamo:

Il linguaggio C è un **linguaggio tipizzato**.
Il concetto di tipo di dato viene introdotto per raggiungere **due obiettivi**:

- Esprimere in modo sintetico la loro **rappresentazione in memoria**, e un insieme di **operazioni ammissibili**.
- Permettere di effettuare **controlli statici** (al momento della compilazione) sulla **correttezza del programma**.

⚠ **In C i tipi sono stretti, non laschi come in altri linguaggi: in ogni variabile bisogna specificare il tipo e avviene un rigido controllo sulle operazioni possibili.**

Tipi primitivi

Di cosa parliamo:

Un **elaboratore** è un manipolatore di simboli (segni).
L'architettura fisica di ogni elaboratore è intrinsecamente capace di trattare **vari domini di dati**, detti **tipi primitivi**.

Che sono:

- Dominio dei **numeri naturali** e **interi**
- Dominio dei **numeri reali** (con qualche approssimazione)
- Dominio dei **caratteri**
- Dominio delle **stringhe di caratteri**

> ⚠ **Dominio valori booleani (true false) è presente nel dominio dei numeri interi (ciò che 0 è falso, il resto vero)**

Link:
Linguaggio tipizzato

Gli operatori

Cosa sono:

Ogni linguaggio introduce un **insieme di operatori** che permettono di **aggregare altre espressioni** (operandi) per formare **espressioni composte** con riferimento a diversi domini / tipi di dato (numeri, testi, ecc.)

Classificazione operatori:

Due criteri di classificazione:

- In base al **tipo** degli operandi
- In base al **numero** degli operandi

TIPO	NUMERO
- Aritmetici - Relazionali - Logici - Condizionali - ecc...	- Unari - Binari - Ternari - ecc...

Ordine di valutazione degli operandi:

In C gli operandi non hanno priorità di per sé, ma la libertà è lasciata ai compilatori, dunque è fondamentale l'uso delle parentesi.

Cosa non scrivere mai:

```
x = 3;
(x = 5) + x; //l'espressione cambia significato in base a quale operatore analizzo per primo
```

Overloading semantico:

L'overloading semantico, , è una caratteristica di un linguaggio di programmazione che permette di definire **più funzioni o operatori** con lo **stesso nome**, ma con **firme** (cioè tipi e numero di argomenti) **diverse**, consentendo di fornire comportamenti differenti in base **al contesto di chiamata**.

Link:
C

Operatori di assegnamento compatti

Zucchero sintattico:

Il C introduce una **forma particolare** di assegnamento che ingloba anche un'operazione:

```
<identificatore> OP= <espressione>

//che equivale a:

<identificatore> = <identificatore> OP <espressione>
```

Esempi:

```
k += j --> k = k + j
k *= a + b --> k = k * (a+b)
```

Operatori aritmetici

Quali sono:

Operazione	Operatore	C
Inversione di segno	unario	-
Somma	Binario	+
Differenza	Binario	-
Moltiplicazione	Binario	*
Divisione tra interi	Binario	/
Divisione tra reali	Binario	/
Modulo (tra interi)	Binario	%

Link:
Gli operatori

Operatori relazionali

Cosa sono:

Sono **operatori binari** per il confronto dei valori di due espressioni:

Relazione	C
Uguaglianza	==
Diversità	!=
Maggiore di	>
Minore di	<
Minore o uguale a	<=
Maggiore o uguale a	>=

⚠ **Non esistendo il tipo boolean, in C le espressioni relazionali denotano un valore intero:**

- `0` denota falso (condizione non verificata)
- `1` denota vero (condizione verificata)

ⓘ **3 > 2 > 1 è (3>2)>1 cioè 1>1 = 0 cioè falso**

Link:

Gli operatori

Specificatori e modificatori di tipo

Cosa sono:

- **Specificatore:** elemento del linguaggio di programmazione che indica il **tipo** di un elemento (come una variabile o una funzione).
- **Modificatore**: parola chiave che si aggiunge ai tipi di dati per specificare meglio la loro natura o il modo in cui vengono gestiti, come la dimensione in memoria, il segno (positivo/negativo) o l'intervallo di valori supportato.

Elenco in C:

- Specificatori di tipo:

```
void //indefinito o non specificato
char //caratteri
float //numeri reali (decimali) a precisione singola
double //numeri decimali a precisione doppia
```

- Modificatori di tipo:

```
short //rappresenta un numero con il numero minore di byte
long //rappresenta un numero con un maggiore numero di byte
signed //dato con segno
unsigned //dato senza segno
```

Quanti bit sono usati per un tipo?

Dipende dal compilatore
Ma abbiamo alcune regole:

- `short int` almeno 16 bit
- `int` a discrezione del compilatore ma vale sempre:

```
    sizeof(int) >= sizeof(shortint)
```

- `long int` almeno 32 bit e vale sempre:

```
sizeof(long int) >= sizeof(int)
```

- `float` nessun limite minimo, ma tipicamente almeno 32 bit

- `double` nessun limite minimo, ma tipicamente almeno 64 bit

Link:
Tipi primitivi

Alfabeto e Lessico

Nel C ci sono almeno 96 simboli tra lettere, numeri, spazi e caratteri grafici.

Categorie lessicali:

Parole chiave:

- Ad es `auto, continue, else` ...
- NO BREAK, CONTINUE E GOTO

Costanti:

Costanti

Una **costante** è un valore che **non può cambiare durante l'esecuzione del programma**.
È l'opposto di una variabile: la variabile può essere modificata, la costante no

Identificatori:

- Si usano per definire variabili e funzioni
- Case sensitive

> ⚠ **Deve iniziare con una lettera o con l'underscore (identificatori speciali di solito)**

Commenti:

- Sequenza di caratteri racchiuse fra i delimitatori // o /* */

Parole chiave

Le **parole chiave** sono i simboli terminali del C, costituiscono cioè i comandi fissi di questo linguaggio.
Sono 32:

```
auto      double    int       struct
break     else      long      switch
case      enum      register  typedef
char      extern    return    union
const     float     short     unsigned
continue  for       signed    void
default   goto      sizeof    volatile
do        if        static    while
```

Non usare se non indispensabili le seguenti parole chiave:

```
break;
continue;
goto;
```

Link:
Alfabeto e Lessico

Costanti

Una **costante** è un valore che **non può cambiare durante l'esecuzione del programma**.
È l'opposto di una variabile: la variabile può essere modificata, la costante no.
Es:

```
const int giorni_settimana = 7;
```

Identificatori

Un **identificatore** in C è il **nome** che il programmatore assegna a un elemento del programma:

- variabile
- funzione
- costante simbolica
- tipo definito dall'utente (`typedef` , `struct` , `enum` , …)

Serve per **riferirsi a quell'oggetto** nel codice.

Es:

```c
int eta;           // identificatore: eta
float _media;      // identificatore: _media
char nomeStudente[20]; // identificatore: nomeStudente

int main() {       // identificatore: main
    return 0;
}
```

Link:
Alfabeto e Lessico

Commenti

I **commenti in C** sono parti di testo dentro al codice che **non vengono eseguite dal compilatore**, servono solo al programmatore per spiegare, annotare o rendere più leggibile il codice.

Es:

```c
// Questo è un commento a riga singola
int x = 10;  // Inizializzo x a 10
```

```c
/* Questo è un commento
   su più righe */
int y = 20;
```

Link:
Alfabeto e Lessico

Compilatore

Cos'è:

Strumento che traduce il programma sorgente in programma oggetto.

Ha due compiti principali:

- Analisi
- Sintesi

Link:
Compilato

Analisi

Cos'è:

Nel corso dell'analisi del file sorgente il compilatore verifica la **correttezza sintattica** e **semantica** del programma.

Si divide in:

- Analisi lessicale
- Analisi sintattica
- Analisi semantica

Lessicale

Cosa fa:

Verifica che i simboli utilizzati siano legali, cioè che appartengano all’alfabeto.

Semantica

Cosa fa:

Verifica i **vincoli** imposti dal **contesto**
Tutto corretto ma si sbaglia il senso.

Es:
"Simulatore bancomat" --> quando prelevi ti aggiungono soldi invece di toglierli --> tutto ok ma non era quello che si voleva ottenere.

Sintesi

Cos fa:

Trasforma il codice da linguaggio di alto livello a linguaggio di basso livello.

Si compone di:

- Allocazione della memoria e dei registri
- Generatore di codice
- Ottimizzatore di codice

Allocazione della memoria e dei registri

Cos'è:

Durante la compilazione, il compilatore deve decidere **dove salvare le variabili**. Quelle usate più spesso vengono messe nei **registri** della CPU (pochissimi ma velocissimi). Le altre vanno allocate in **memoria**, che è più lenta ma abbondante.

È il primo processo della sintesi

Generatore di codice

Cosa fa:

Trasla la forma intermedia in linguaggio assembler o macchina.

Avviene dopo l'allocazione della memoria e dei registri.

Ottimizzatore di codice

Cosa fa:

Ottimizza il codice scritto a basso livello. Non è fondamentale.

Agisce dopo il generatore di codice.

Linker

Il linker collega il programma oggetto alle librerie incluse nel programma e genera il File eseguibile.

Se non si includono le librerie il linker non serve.

Link:
File oggetto

Librerie

Le **librerie** sono insiemi di codice precompilato (funzioni e definizioni) che puoi riutilizzare nei tuoi programmi. Permettono ai programmatori di evitare di riscrivere codice comune e complesso per attività standard.

Es:

```c
// 1. Inclusione del File Header
#include <stdio.h>

int main() {
    // 2. Uso della funzione dichiarata in stdio.h
    printf("Ciao, Mondo! Questo è un esempio di libreria C.\n");

    // Dichiarazione di una variabile
    int numero = 42;

    // Uso della funzione per formattare l'output
    printf("Il numero di esempio è: %d\n", numero);

    return 0;
}
```

main

L'unica parte indispensabile è il main, che è a sua volta una particolare unità di traduzione (funzione). La sua struttura è:

```
<main> ::=
int main() {[<dichiarazioni-edefinizioni>]
[<sequenza-istruzioni>]
}
```

Si divide dunque in:

- Dichiarazioni e definizioni
- Sequenza di istruzioni

Potere espressivo

Cos'è:

Il potere espressivo di un linguaggio è caratterizzato da:

- Quali tipi di dati consente di rappresentare (direttamente o tramite definizione dell'utente)
- Quali istruzioni di controllo mette a disposizione (quali operazioni e in quale ordine di esecuzione)

Variabili

Definizione:

- Astrazione di una cella di memoria
- Formalmente è un simbolo associato a un indirizzo fisico (L-value)

L-value	R-value
1328	4

simbolo	indirizzo
X	1328

La **definizione** è composta da:

- nome della variabile (identificatore)
- tipo di valori (R-value) che possono essere denotati alla variabile

Le variabili sono note solo nel blocco nel quale vengono dichiarate.

Scope della variabile: **porzione di programma** in cui quella variabile è **visibile** e quindi può essere utilizzata.

Tempo di vita di una variabile: è l'intervallo di tempo in cui la variabile rimane valida.

Definizione di una variabile:

Es:

```
<tipo> <identificatore>;
int x; /* x deve denotare un valore intero */
float y; /* y deve denotare un valore reale */
char ch; /* ch deve denotare un carattere */
```

Inizializzazione di una variabile:

Contestualmente alla **definizione** è possibile **specificare un valore iniziale** per una variabile (altrimenti il suo contenuto è non definito).

Espressione formale:

```
<tipo> <identificatore> = <espr> ;
```

Esempio:

```
int x = 32;
double speed = 124.6;
```

Blocco

Sintassi:

```
<blocco> ::= {
[ <dichiarazioni e definizioni> ]
{ <istruzione> }
}
```

Nota:

• Il **campo di visibilità** dei simboli del blocco è **ristretto al blocco stesso**.
• I blocchi possono essere **innestati**.
• Dopo un blocco **non occorre il punto e virgola** (esso termina le istruzioni semplici, non separa istruzioni).

Esempio:

```
 /* programma che, letti due numeri a terminale, ne stampi la somma /*

#include <stdio.h>
int main() {/* INIZIO BLOCCO */
int X,Y;
printf(“Inserisci due numeri ”); scanf("%d%d",&X,&Y);
printf("%d",X+Y);
} /* FINE BLOCCO */
```

Link:
Istruzioni di controllo

Do-while

Sintassi:

```
<do-while> ::=
do <istruzione> while(<condizione>);
```

È una variante della precedente: la condizione viene **solo dopo** aver eseguito l'istruzione.

Se la condizione **è falsa**, l'iterazione viene comunque eseguita **almeno una volta**.

Esempio:

```c
#include <stdio.h>

int main() {
    int numero;

    // L'istruzione 'do' esegue il blocco di codice al suo interno
    do {
        printf("Inserisci un numero maggiore di 10: ");
        scanf("%d", &numero);

        // Se il numero è minore o uguale a 10, stampa un messaggio
        if (numero <= 10) {
            printf("Numero troppo basso! Riprova.\n");
        }

    } while (numero <= 10); // La condizione viene testata alla fine.

    // Il codice esce dal ciclo solo quando 'numero' è > 10
    printf("\nComplimenti! Hai inserito il numero %d. Uscita dal
ciclo.\n", numero);

    return 0;
}
```

Link:
Istruzioni di iterazione

Equivalenza While-For

Come si struttura:

```
for (e1; e2; e3) <istruzione>

equivalente a:

e1;
while (e2)
{<istruzione>
e3;}
```

Espressioni concatenate

Cosa sono:

Un’ espressione concatenata è introdotta dall’operatore di concatenazione (la virgola):

```
espr1, espr2, …, esprN
```

Tutte le espressioni vengono valutate da sinistra a destra e l’espressione globale esprime il valore denotato da `esprN`.

Es:

Supponiamo che:

- `i` valga `5`
- `k` valga `7`

Allora l’espressione: `i + 1, k - 4` denota il valore denotato da `k-4`, cioè `3`.

Link:
Espressioni

Espressioni condizionali

Cosa sono:

Un'espressione condizionale è introdotta dall'operatore ternario:

```
condiz ? espr1 : espr2
```

L'espressione denota in base al valore della espressione `condiz`:

- O il valore denotato da `espr1`
- O quello denotato da `espr2`

Dunque:

- Se `condiz` è vera, l'espressione nel suo complesso denota il valore denotato da `espr1`
- Se `condiz` è falsa, l'espressione nel suo complesso denota il valore denotato da `espr2`

Esempi:

```
3 ? 10 : 20
```

Denota sempre 10 (3 è sempre vera).

```
x ? 10 : 20
```

Denota 10 se x è vera (diversa da 0), oppure 20 se x è falsa (uguale a 0).

```
(x>y) ? x : y
```

Denota il maggiore fra x e y.

Link:
Espressioni

For

Sintassi:

In genere si usa quando **è noto quante volte** il ciclo dovrà essere eseguito (contatore).

```
<for> ::=
for (<espr-iniz>;<cond>;<espr-modifica>)
<istruzione>
```

- **Espressione di inizializzazione**:
  valutata **una e una sola volta** prima di iniziare l'iterazione.
- **Condizione**:
  valutata a ogni **interazione**, per decidere se proseguire (come in un while).
- **Espressione di modifica**:
  valutata **a ogni interazione**, dopo aver eseguito l'istruzione.

Esempio:

```c
#include <stdio.h>
int main() /* Media di N voti*/
{ int sum,voto,N,i;
    float media;
    printf("Quanti sono i voti?");
    scanf("%d",&N);
    sum = 0;
    for(i=1; i<=N; i++)
        { printf("Dammi il voto n.%d:",i);
        scanf("%d",&voto);
        sum=sum+voto;
        }
    media=((float)sum)/N;
    printf("Risultato: %f",media);
}
```

Link:
Istruzioni di iterazione

Istruzione di selezione

Sintassi:

```
<scelta> ::= if (<cond>) <istruzione1>
[ else <istruzione2> ]
```

La parte `else` è opzionale: se omessa, in caso di condizione falsa si passa subito all'istruzione che segue `if`.

Es:

```
if (n > 0) { /* inizio blocco */
a = b + 5;
c = a;
} /* fine blocco */
else n = b;
```

Link:
Istruzioni di controllo

Istruzioni di iterazione

Sintassi:

```
<iterazione> ::=
<while> | <for> | <do-while>
```

Le **istruzioni di iterazione**:

• Permettono di specificare che un blocco **sia eseguito ripetutamente** fino a che le condizioni di permanenza nel ciclo siano soddisfatte.

• Hanno **un solo punto di ingresso** e **un solo punto di uscita** nel flusso del programma, perciò possono essere interpretate come **una singola azione** in una computazione sequenziale.

• Possono essere convertiti **uno nell'altro** (ne basterebbe in teoria uno solo).

Link:
Istruzioni di controllo
Equivalenza While-For

Istruzioni di salto

Cosa sono:

• Permettono di alterare il flusso di esecuzione di una sequenza di istruzioni.

• Il Linguaggio C ha **4 istruzioni di salto**:

- `break` : forza l'uscita da un ciclo e dallo switch
- `continue` : forza il riavvio della prossima iterazione di un ciclo
- `goto` : forza il salto incondizionato ad un punto della funzione (da NON usare)
- `return` : fa tornare il controllo al chiamante nelle funzioni

Attenzione!

- Il loro utilizzo nella programmazione è controverso
- Programmi di difficile lettura.
- •Ad eccezione di return e break nello switch non vanno usate.
- Cattivo stile di programmazione (spaghetti code).
- Tutto può essere scritto utilizzando la programmazione strutturata pura.

Link:
Istruzioni di controllo

Istruzioni di selezione multipla

Sintassi:

```
<scelta-multipla> ::=
switch (selettore) {
case <label1>:
{case <labeli>:} <istruzioni> [break;]
case <labelj>:
{case <labeln>:} <istruzioni> [break;]
…
[ default : <istruzioni>]
}
```

Il valore di **selettore** (espressione intera) viene confrontato con le **etichette** (costanti dello stesso tipo del selettore): l'esecuzione prosegue **dal ramo corrispondente** (se esiste)

Se nessuna etichetta corrisponde, si prosegue con il ramo **default** se esiste, altrimenti non si fa niente.

> ⚠ **I vari rami non sono mutuamente esclusivi: imboccato un ramo, si eseguono anche tutti i rami successivi a meno che non ci sia il comando break a forzare esplicitamente l'uscita**

Esempio:

```c
int main() {
    int giorno = 3; // La nostra espressione da valutare

    switch (giorno) {
        case 1:
            printf("Lunedì\n");
            break;
        case 2:
            printf("Martedì\n");
            break;
        case 3:
            printf("Mercoledì\n"); // Esecuzione inizia qui
            break; // Esecuzione termina qui e salta dopo la graffa di
```

```
chiusura
        case 4:
            printf("Giovedì\n");
            break;
        case 5:
            printf("Venerdì\n");
            break;
        default:
            printf("Giorno non valido (o weekend)\n");
            break; // Il break è opzionale qui, ma è buona pratica.
    }
    return 0;
}
```

```
Output: Mercoledì
```

Link:
Istruzioni di controllo

Istruzioni

Definizione:

Le **istruzioni** esprimono azioni che, una volta eseguite, possono comportare una **modifica permanente** dello stato interno del programma o del mondo circostante.

Le **strutture di controllo** permettono di aggregare **istruzioni semplici** in **istruzioni più complesse**.

Un'**istruzione C** è espressa dalle seguenti produzioni:

```
<istruzione> ::= <istruzione-semplice>
<istruzione> ::= <istruzione-di-controllo>
<istruzione-semplice> ::= <espressione>;
```

Istruzioni semplice:

Qualsiasi espressione seguita da un punto e virgola è **un' istruzione semplice**.

Es:

```
x = 0; y = 1;/* due istruzioni */
x = 0, y = 1;/* una istruzione */
k++;
3; /* non fa nulla */
; /* istruz. vuota*/
```

Istruzioni di controllo:

Sono particolari istruzioni tipiche dei **linguaggi "imperativi"** che eseguono istruzioni sotto **determinate condizioni**.

Un'istruzione di controllo può essere:

- Un'**istruzione composta sequenziale** (blocco)
- Un'**istruzione condizionale** (selezione)
- Un'**istruzione di iterazione** (ciclo)

Le istruzioni di controllo sono alla base della programmazione strutturata (Dijkstra, 1969)

Bohm e Jacopini (1966): queste tre strutture di controllo sono sufficienti per definire tutte le funzioni computabili

Link:

C

Operatori infissi, prefissi e post fissi

Tre possibili scelte:

Abbiamo tre scelte di posizionamento degli operatori:

• **Prima $\rightarrow$ notazione prefissa**
Esempio: $+3\ 4$

• **Dopo $\rightarrow$ notazione postfissa**
Esempio: $3\ 4+$

• **In mezzo $\rightarrow$ notazione infissa**
Esempio: $3\ +\ 4$

In C adottiamo la **notazione infissa**, questo porta a dei **problemi di priorità** degli operatori. Per stare sicuri dobbiamo **sempre mettere delle parentesi** per specificare al compilatore come si deve comportare.

Link:
Gli operatori

Operatori di incremento e decremento

Uso:

Gli operatori di incremento e decremento sono usabili in due modi:

- Come **pre-operatori**: `++v` prima incremento e poi uso nell'espressione.
- Come **post-operatori**: `v++` prima uso nell'espressione poi incremento.

⚠**Bisogna stare molto attenti con l'ordine in cui agiscono gli operatori!**

Link:
Gli operatori

Operatori logici

Quali sono:

Connettivo logico	Operatore	C
not	unario	!
and	binario	&&
or	binario	|||

Nota:

- Anche le espressioni logiche denotano un **valore intero** (1 se vero 0 se falso)

Valutazione in corto-circuito:

– A differenza degli altri operatori hanno **un preciso ordine di valutazione** da sinistra a destra
– La valutazione dell'espressione **cessa** appena si è in grado di **determinare il risultato**
– Il secondo operando è valutato **solo se necessario**

Es:

```
22 || x
```

Già **vera in partenza** perché 22 è vero.

```
0 && x
```

Già falsa in partenza perché 0 è falso.

```
a && b && c
```

Se `a && b` è falso, il secondo `&&` non viene neanche valutato.

```
a || b || c
```

Se `a||b` è vero, il secondo `||` non viene neanche valutato.

Link:
Gli operatori

Regole di visibilità

Cosa sono:

Esistono delle **regole di visibilità** per gli identificatori (nomi di variabili, di funzioni, costanti) che definiscono in quali parti del programma tali identificatori **possono essere usati**.

In un programma esistono **diversi ambienti**:
• Area globale
• Il main
• Ogni singola funzione
• Ogni blocco

Quali sono:

• Un identificatore **NON è visibile** prima della sua dichiarazione.

• Un identificatore definito in un ambiente è visibile in tutti gli ambienti **in esso contenuti**.

• Se in un ambiente sono visibili **due definizioni** dello stesso identificatore, la definizione valida è quella dell'ambiente **più vicino al punto di utilizzo**.

• In ambienti diversi si può definire **lo stesso identificatore** per denotare due oggetti diversi

• In ciascun ambiente un identificatore può essere **definito una sola volta**.

Link:
Blocco

While

Sintassi:

```
<while> ::=
while(<condizione>) <istruzione>
```

- L'istruzione viene ripetuta per tutto il tempo in cui **la condizione rimane vera**.
- Se la condizione è falsa, l'iterazione **non viene eseguita neppure una volta**.
- In generale, **NON è noto** quante volte l'istruzione sarà ripetuta

⚠ **Prima o poi, direttamente o indirettamente, l'istruzione deve modificare la condizione: altrimenti, CICLO INFINITO.**

Esempio:

```
/* Calcolo del fattoriale di un numero N */
#include <stdio.h>
int main()
{   int F, N, I;
    F=1; /* inizializzazione del fattoriale*/
    I=1; /* inizializzazione del fattoriale*/
    printf(“Dammi N:”);
    scanf("%d",&N);
    while (I <= N)
    {F = I*F;
    I = I+1;
    }
   printf(“Il fattoriale è %d”, F);
}
```

Link:
Istruzioni di iterazione

Direttive

Definizione e caratteristiche:

- Le **direttive** sono tutte le righe di codice di un file sorgente che iniziano con **#**.
- Sono istruzioni che verranno eseguite dal **precompilatore**
- Non sono istruzioni in C e non ne hanno la sintassi.
- Le direttive vengono soppresse una volta elaborate (il compilatore non ne ha visibilità)

In concreto:

Possono:

- Includere altre porzioni di testo:
  - `#include nomefile`
- Effettuare ricerche e sostituzioni:
  - `#define identificatore testo`
- Inserire o sopprimere parti del testo:

```
#ifdef cond          #ifndef cond
...testo...          ...testo...
#endif               #endif
```

Link:
Define
Include

Define

Definizione:

È una **direttiva**

Permette di definire delle **macro**

Definisce una regola di **ricerca e sostituzione**: ogni occorrenza di **Identificatore** verrà sostituita da **testo** (a partire dal separatore bianco fino alla fine della riga)

```
#define IDENTIFICATORE testo
```

Lo scopo è quello di **definire costanti simboliche**.

> **Per convenzione, IDENTIFICATORE si tende a scrivere con tutte maiuscole**

Le macro:

La regola di **ricerca e sostituzione**introdotta dalla direttiva `#define` si chiama **macro**.

Regole semplici, come le precedenti:

```
#define MAX 10
#define RADICEDIDUE 1.4142F
```

Definiscono **macro semplici**.
La direttiva `#define` permette però anche di definire **regole più complesse**, che vanno sotto il nome di **macro parametriche**.

Esempio:

Prima del pre-processing:

```
#define RADICEDI2 1.4142F
int main()

{   float lato = 18;
    float diagonale = lato * RADICEDI2;
}
```

Dopo il pre-processing:

```
int main()
{
float lato = 18;
float diagonale = lato * 1.4142F;
}
```

Efficienza

Definizione:

> Determinazione di un algoritmo che consumi **poche risorse** (in termini di uso di memoria e di tempo di CPU principalmente) e sua codifica **efficace e “sostenibile”** (minime risorse utilizzate, senza penalizzare modularità e leggibilità)

Include

Definizione:

Include il contenuto del **file specificato** esattamente nella posizione in cui si trova la **direttiva stessa**.

```
#include <libreria.h>
#include “miofile.h”
```

La differenza tra le due scritture impatta solo su dove –in quale direttorio- il file incluso verrà cercato dal preprocessore, rispettivamente nelle directory standard con parentesi angolari, nella corrente con doppi apici.

Modularità

Definizione:

> Divide et impera

Un problema **complicato** va scomposto in **piccoli problemi più semplici**.

Le singole **sotto-parti** di soluzioni vanno costruite in modo che possano essere **riusabili in altri contesti**.

Concetto chiave perché:

- Si evita di **reinventare la ruota** tutte le volte.
- Si fa affidamento su librerie **ampiamente diffuse**, quindi testate e affidabili.
- Moduli scritti per utilizzo da parte di una **comunità**.

Ordine

Definizione:

È fondamentale che un programma sia leggibile:

- **Facile comprensione del codice** da parte di chi non l'ha scritto ma vorrà manutenerlo
- **Chiara strutturazione**

Per aumentare la leggibilità:

- Regole di naming
- Regole di strutturazione del codice

Principi di programmazione

Quali sono?:

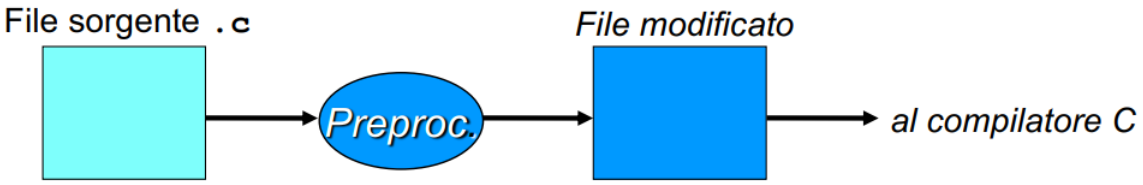
- Efficienza
- Modularità della soluzione
- Ordine e leggibilità

Link:
C
Efficienza
Modularità
Ordine

Preprocessore

Definizione:

- Unità che agisce **prima del compilatore** e processa direttamente il file sorgente.
- **Esegue** direttive possibilmente contenute nel file sorgente.
- Il compilatore riceve una versione "riveduta" del file sorgente.



Attenzione:

- Il preprocessore **non è un compilatore**:
  - Non conosce il linguaggio C:
    - Non può interpretarne le istruzioni
    - Non può verificare la correttezza del programma
- Agisce **esclusivamente sul testo** del programma:
  - Potrebbe manipolare qualunque testo…
  - …anche codice sorgente di altri linguaggi!

Link:
C

Regole di naming

Definizione:

- I nomi di variabili e funzioni devono avere **nomi autoesplicativi**
- Usare il **Camel Casing** sia per le variabili sia per le funzioni:

```
rowIndex, mangiaFagioli, mediaDivertente
```

Binding ed Environment

Di cosa parliamo:

- **Binding (Legame):** È l'associazione tra un **simbolo** (come il nome di una variabile, `x`) e i suoi **attributi** (come il suo tipo `int`, la sua posizione in memoria e il suo valore attuale `5`). È il "legame" che dà significato al nome.
- **Environment (Ambiente):** È l'**insieme di tutti i binding validi** in un certo punto. Quando sei dentro una funzione C, il tuo ambiente è l'elenco di tutti i nomi che puoi usare: le variabili locali, i parametri della funzione e tutte le variabili e funzioni globali.

Link:
Funzioni

Chiamata di funzione

Espressione:

La chiamata di funzione è un'espressione della forma:

```
<nomefunzione> (<parametri-attuali>)

<parametri-attuali> ::=
[<espressione>] {, <espressione>}
```

> **La lista di parametri attuali può essere vuota se la definizione della funzione indica `lista void` per i parametri formali. Ad esempio: `<nomefunzione>()`**

All'atto **dell'invocazione** di una funzione:
– si crea una **nuova attivazione** (istanza) del servitore
– si alloca la **memoria per i parametri** (e le eventuali variabili locali)
– si trasferiscono i **parametri al servitore**
– si trasferisce il **controllo al servitore**
– si esegue il **codice della funzione**

Funzione main()

Cos'è:

Anche `main()` è una funzione, che in particolare restituisce valore di ritorno `int` e non ha parametri di ingresso (almeno diciamo così per il momento…).

Andrebbe definita: `int main(void)` oppure `main(void)` .
La scrittura `void` può essere omessa nella definizione dei parametri formali (se la funzione non ha parametri di ingresso).

Definizione formale Funzioni

Definizione:

```
<definizione-di-funzione> ::=
<tipoValore> <nome>(<parametri-formali>)
{
    <corpo> //La forma base è return <espressione>;
}
```

parametri-formali:

• O una lista vuota: `void`
• O una lista di variabili (ciascuna con il proprio tipo e separate da virgole) visibili solo entro il corpo della funzione

tipo-valori:

• Deve coincidere con **il tipo del valore restituito dalla funzione**
• Se non indicato esplicitamente, si sottintende `int`
• Se non si desidera valore di ritorno, `void` .

Link:
Funzioni

Funzioni

Cosa sono:

Spesso può essere utile avere la possibilità di costruire **“nuove istruzioni”** che risolvano parti specifiche di un problema, scomponendolo in sotto-parti.

Rendono **più chiaro il codice** e **più facile il suo riutilizzo e manutenzione**.

Dalla definizione matematica prendono in ingresso **un insieme di valori** e producono **un valore** in uscita.

Cosa permettono:

Una funzione permette di:
• **dare un nome** a una espressione
• rendere tale espressione **parametrica**

Es (pseudo-C):

```
float f(){ 2 + 3 * sin(0.75); }
float f1(int x) { 2 + x * sin(0.75); }
```

Link:
C

Interfaccia Funzione

Cos'è:

L'interfaccia (o firma o signature o prototipo) di una funzione comprende:

- Nome della funzione
- Lista dei parametri
- Tipo del valore da essa denotato
- Esplicita il contratto di servizio fra cliente e servitore

Cliente e servitore comunicano quindi mediante:

- I **parametri trasmessi** dal cliente al servitore all'atto della chiamata
- Il **valore restituito** dal servitore al cliente

Esempio:

```
int max (int x, int y )
{
    if (x>y) return x ;
    else return y;
}
```

Limiti

A causa della **mancanza della possibilità** di mandare alle funzioni dei parametri "by reference", l'uso delle funzioni in C ha numerosi limiti, tra cui:

- Consente di restituire al cliente solo valori di tipo (relativamente) **semplice**
- Non consente di restituire **collezioni di valori**
- Non consente di scrivere componenti software il cui obiettivo sia diverso **dal calcolo di una espressione**

Modello Cliente-Servitore

Servitore:

> Un qualunque ente computazionale capace di **nascondere la propria organizzazione interna** presentando ai clienti una precisa interfaccia **per lo scambio di informazioni**.

Cliente:

> Qualunque ente in grado **di invocare uno o più servitori** per svolgere il proprio compito.

Una Funzione è un servitore:

- Passivo
- Che serve **un cliente per volta**
- Che può **trasformarsi in cliente** invocando altre funzioni o se stessa
- Una funzione è **un servitore dotato di nome** che incapsula le istruzioni che realizzano un certo servizio

Il cliente nella Funzione:

- Il cliente chiede al servitore di svolgere il servizio:
- Chiamando tale servitore (per nome)
- Fornendogli le necessarie informazioni

Nel caso di una funzione, cliente e servitore comunicano mediante l'interfaccia della funzione.

Parametri

Parametri Formali:

- Sono specificati nella **definizione del servitore**
- Indicano che cosa **il servitore si aspetta** dal cliente

Parametri attuali:

- Sono trasmessi dal cliente **all'atto della chiamata**
- Devono corrispondere ai parametri formali in **numero, posizione e tipo**
- Possono essere il valore di un **qualunque tipo di espressione coerente coi parametri formali**.
- L'**ordine di valutazione** dei parametri o argomenti della funzione **non è specificato dal C**.

Es:

```
int max (int x, int y ) //x e y sono parametri formali
{
    if (x>y) return x;
    else return y;
}
```

```
main()
{
    int z = 8;
    int m1,m2;
    m1 = max(z,4); //z e 4 sono i par. attuali
    m2 = max(5,z);
}
```

Chiamata di funzione:

Il legame tra **parametri attuali** e **parametri formali** viene effettuato al momento della chiamata, in **modo dinamico**.
Tale legame:

- Vale **SOLO** per l'invocazione corrente
- Vale **SOLO** per la durata della funzione

Link:

Funzioni

Passaggi di parametri

In generale un parametro può essere trasferito **dal cliente ad servitore**:

- Per **valore** o copia (by value)
- Per **riferimento** (by reference)

By value:

Si trasferisce una copia del valore del **parametro attuale**:
Ogni azione compiuta dal servitore sul parametro ricevuto è **locale al servitore**.

By Reference:

Si trasferisce **un riferimento** al parametro attuale.
Ogni azione fatta sul parametro ricevuto è in realtà fatta **sulla variabile del cliente**.

> ⚠ **In C, i parametri sono trasferiti sempre e solo by value**

Dunque:
• **È impossibile** usare un parametro per trasferire informazioni **verso il cliente**.
• Per trasferire un'informazione al cliente si **sfrutta il valore di ritorno** della funzione.

Return

Return:

L'istruzione `return` provoca la **restituzione del controllo al cliente**, unitamente al valore dell'espressione che la segue.

⚠ **Eventuali istruzioni successive a `return` non saranno mai eseguite**

Esempio:

```
int max (int x, int y )
{
    if (x>y) return x;
    else return y;
}
```

Link:
Funzioni

Vita di una funzione

La vita:

All'atto della chiamata **l'esecuzione del cliente viene sospesa** e il **controllo passa al servitore**.

Il servitore “vive” **solo per il tempo necessario** a svolgere il servizio.

Al termine, il servitore **libera le risorse eventualmente allocate** e l’esecuzione **torna al cliente**.

Link:
Funzioni

Funzioni ricorsive

Cosa intendiamo:

Una funzione matematica **è definita ricorsivamente**
quando nella sua definizione **compare un riferimento a se stessa.**

La ricorsione consiste nella possibilità di definire una funzione in termini di se stessa.

L'induzione:

È basata sul principio di **induzione matematica**:
– Se una proprietà P vale per $n = n_0$ (caso base)
– e si può provare che, assumendola valida per n, allora vale per $n + 1$ allora $Pvaleperogni\$n \geq 0$

Operativamente:

Operativamente, risolvere un problema con un approccio ricorsivo comporta:

1. Identificare un **"caso base"**, con soluzione nota per cui la funzione **termina subito**
2. Di riuscire a esprimere la soluzione **al caso generico** n in termini dello stesso problema in uno o più **casi più semplici** ($n - 1$, $n - 2$, etc.).

Es:

```c
#include <stdio.h>


int fact(n) {
    if (n == 0 || n == 1) {
        return 1;
    }
    else {
        return (fact(n - 1) * n);
    }
}
int main() {
    int num;

    printf("Inserisci un numero intero: ");
    scanf_s("%d", &num);
```

```
    printf("\nIl fattoriale di %d: %d", num, fact(num));
    printf("\n");
}
```

Operatore di dereferenziamneto

```
*
```

Se a è l'indirizzo di una variabile, $*a$ denota tale variabile:

```
x = *a
```

Link:
Puntatori

Operatore di estrazione di indirizzo

```
&
```

Consente di ricavare l'indirizzo di una variabile.

Es:

```c
#include <stdio.h>

int main() {
    int a = 1;
    printf("%d", &a); //Restituisce l'indirizzo della variabile
}
```

Link:
Puntatori

Procedure

Cosa sono:

Una procedura permette **di dare un nome ad un istruzione**, rendendola parametrica.
Non denota un valore, quindi **non c'è tipo di ritorno**.

Ha la stessa struttura di una funzione, salvo **il tipo di ritorno che è void**.

> **L'istruzione return in una procedura provoca solo la restituzione del controllo al cliente e non è seguita da un'espressione da restituire. Non è dunque necessaria se la procedura termina "spontaneamente" a fine blocco**

Non esistendo valore di ritorno, cliente e servitore comunicano solo:

- mediante **parametri**
- mediante **aree globali**

Link:
C

Processo iterativo vs ricorsivo

I due processi:

I due processi sono in realtà interscambiabili, ma hanno delle differenze dal punto di visto strutturale e di performance.

In particolare:

Nei processi iterativi:

Il risultato viene sintetizzato **“in avanti”**.
La caratteristica fondamentale di un processo computazionale iterativo è infatti che a ogni passo è disponibile **un risultato parziale**.

Nei processi ricorsivi:

Il risultato viene sintetizzato **a partire dalla fine**, perché prima occorre arrivare al caso “banale”.
Dunque fino a quando non si è arrivati al caso banale **nulla è disponibile**.

Link:
Funzioni ricorsive

Puntatori

Definizione:

Un puntatore è il costrutto linguistico introdotto dal C (e da altri linguaggi) come **forma di accesso alla macchina sottostante** e in particolare **agli indirizzi di variabili**.

Un **"tipo puntatore a T"** è un tipo che denota **l'indirizzo di memoria** di una variabile di tipo T.

Un **"puntatore a T"** è una variabile di "tipo puntatore a T" che può **contenere l'indirizzo di una variabile di tipo T**.

```
<tipo> * <nomevariabile> ;
```

Esempio di utilizzo:

```
void scambia(int* a, int* b)
{
int t;
t = *a;
*a = *b;
*b = t;
}
int main(){
int y=5, x=33; scambia(&x, &y);
}
```

⚠ **Quando un puntatore è usato per realizzare il passaggio per riferimento, la funzione non dovrebbe mai alterare il valore del puntatore.**

Record di attivazione

Cos'è:

Quando una funzione viene invocata, viene creata una **struttura dati** contente i **binding dei parametri e degli identificatori** definiti localmente alla funzione.

Questa struttura dati viene chiamata **Record di Attivazione**.

Il mondo della funzione:

Contiene tutto ciò che **ne caratterizza l'esistenza**:
– **i parametri ricevuti**
– **le variabili locali**
– **l'indirizzo di ritorno** (Return Address RA):
- indica il punto a cui tornare (nel codice del cliente) al termine della funzione, per permettere al cliente di proseguire una volta che la funzione termina
– un collegamento al **record di attivazione del cliente** (Dynamic Link DL)

Altre caratteristiche del RA:

• Rappresenta il "mondo della funzione": nasce e muore con essa:

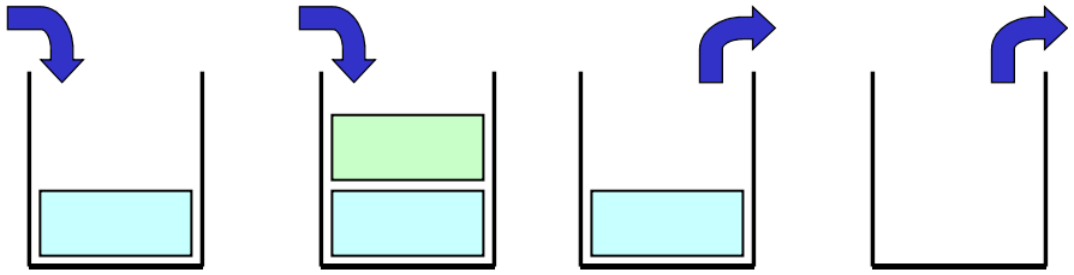
- è creato al momento della invocazione di una funzione
- permane per tutto il tempo in cui la funzione è in esecuzione
- è distrutto (deallocato) al termine dell'esecuzione della funzione stessa
  • Ad ogni chiamata di funzione viene creato un nuovo record, specifico per quella chiamata di quella funzione
  • La dimensione del record di attivazione:
- varia da una funzione all'altra
- per una data funzione, è fissa e calcolabile a priori

Stack di memoria

Cos'è:

L'area di memoria dove vengono allocati i Record di attivazione.

Questa struttura dati è gestita con politica **LIFO** (Last In, First Out).



Variabili globali

Definizione:

Le variabili globali in C:

• Sono allocate nell'**area dati globale** (fuori da ogni funzione)

• Esistono prima della chiamata del `main` (quindi prima dell'esecuzione del programma).

• Il tempo di vita si estende per tutta la durata del programma.

• Sono visibili, previa dichiarazione `extern`, **in tutti i file dell'applicazione**

• Sono inizializzate automaticamente a 0 salvo diversa indicazione

• Possono essere nascoste in una funzione da una variabile locale omonima.

Esempio:

```
int quoziente, int resto;
void dividi(int x, int y){
resto = x % y;
quoziente = x/y;
}
int main(){
dividi(33, 6);
printf("%d%d", quoziente,resto);
}
```

Definizione funzione

Definizione:

La **Definizione** di una funzione dice **come è fatto il componente**.
Essa costituisce l'**effettiva realizzazione** del componente e **non può essere duplicata**.
La compilazione di una definizione genera **il codice oggetto** corrispondente alla funzione.

```
<definizione-di-funzione> ::=
<tipoValore> <nome>(<parametri-formali>)
{
    <corpo> //La forma base è return <espressione>;
}
```

Link:
Funzioni

Dichiarazione funzione

Definizione:
La dichiarazione di una funzione costituisce una specifica delle proprietà del componente:

- Può essere duplicata senza problemi
- Un programma può contenerne più di una
- La compilazione di una dichiarazione non genera alcun codice macchina

```
<dichiarazione-di-funzione> ::=
<tipoValore> <nome>(<parametri>)
```

Link:
Funzioni

File Header

Cosa sono:

Sono **file di intestazione**.
Hanno estensione `.h` e hanno per convenzione nome uguale al file `.c` di cui **fornisce le dichiarazioni**.

> ⚠ **Occhio! Un Header file dovrebbe contenere solo dichiarazioni e non definizioni! (sia di funzioni che di variabili)**

Questo perché renderebbe possibile **problemi di definizione compilata più volte**, generando poi anche **problemi di linking**.

Facciamo un esempio:

Vogliamo realizzare un programma che stampi la somma di due numeri interi tramite la funzione `func1` e vogliamo avere questo programma su più file.
Il programma dunque avrà tre file:

- Il file `func1.c` dove sarà dichiara la funzione `func1`.
- Il file `func1.h` che servirà come collegamento tra cliente e servitore
- Il file `main.c`

Link:
Librerie

Regola Visibilità Funzioni

La regola:

Prima che qualcuno possa invocare una funzione, essa deve essere stata **precedentemente dichiarata** (e non per forza definita!).
Altrimenti si incapperà in un **errore di compilazione**.

```
int fact(int);
int main(){
    int y = fact(3);
    printf(“%d”, y);
    return 0;
}
int fact(int n){
    return (n<=1) ? 1 : n * fact(n-1);
}
```

Link:
Funzioni

Tipi di Dato

Come si differenziano:

In C troviamo i seguenti **tipi di dato**:

Scalari:

Predefiniti:

```
int
char
float
double
```

Definiti dall'utente:

```
enum
```

Strutturati:

Costruttori predefiniti:

```
[]
struct
union
```

Definiti dall'utente.

Puntatori:

```
int* punt
```

Link:
Tipi primitivi
Puntatori

Aritmetica dei puntatori

Consentite:

- **Sottrazione** tra puntatori
- **Addizione** o **Sottrazione** con un intero
- **Assegnazione** (un puntatore può essere assegnato a un altro puntatore dello stesso tipo)

Non consentite:

- **Addizione** tra Puntatori
- **Moltiplicazione** e **Divisione** che coinvolgano puntatori
- **Modulo** tra puntatori

Link:
Puntatori

Array

Definizione:

Un array è una **collezione finita di N variabili** dello stesso tipo, ognuna identificata da un indice compreso fra 0 e $N-1$.

```
<tipo><nome>[<espr-costante-positiva>];
```

È bene definire la dimensione di un array tramite una macro (#define).

Se vi è una inizializzazione esplicita, la dimensione dell'array può essere omessa:

```
int v[] = {43, 12, 7, 86}
```

Note sull'inizializzazione:

Se specifichiamo in inizializzazione **un numero di valori minore** del numero di elementi dell'array, gli elementi esclusi **sono inizializzati a zero**.

Se invece specifichiamo **un numero di valori maggiore** del numero di elementi dell'array, **ciò causerà un syntax error**.

Dimensione Fisica vs. Logica:

Un array è una **collezione finita** di N celle dello stesso tipo disposte in locazioni di **memoria successive e contigue**.
La **dimensione fisica** N è decisa staticamente all'atto della definizione della variabile di tipo array.

Questo non significa che si debbano per forza usare sempre tutte!

La **dimensione logica** di un array può essere **inferiore alla sua dimensione fisica**.

Limiti:

Non è possibile leggere o scrivere un array per intero, ma bisogna compiere ogni operazione per ciascun elemento dell'array.

Inoltre, anche **se due vettori sono dello stesso tipo e uguali**, non è possibile l'assegnamento diretto ma **è necessario farlo per ogni elemento degli array**.

Particolarità degli array:

In C un array è in realtà **un puntatore** che punta a **un'area di memoria pre-**

allocata, di dimensione prefissata.
Dunque **il nome dell'array** è un sinonimo per **il suo indirizzo iniziale**

```
v ≡ &v[0]
```

⚠ **Per questo è impossibile denotare un array nella sua globalità in qualunque contesto.**

Legame puntatori-array:

Come abbiamo detto dunque, un array non è altro che un puntatore che punta alla cella di memoria del suo primo valore.
Siccome però sappiamo che i vari valori degli array si trovano in posizioni di memoria contigue, allora di fatto queste scritture sono equivalenti:

```
*v ≡ v[0]
*(v+1) ≡ v[1]
*(v+i) ≡ v[i]
```

Stringhe

Definizione:

Una stringa di caratteri in C **è un array di caratteri** terminato dal carattere '\0'.

> **Un vettore di N caratteri può dunque ospitare stringhe lunghe al più $N-1$ caratteri, perché una cella è destinata al terminatore '\0'.**

Inizializzazione:

```
char s[4] = {'a', 'p', 'e', '\0'};

oppure

char s[4] = "ape";
(Il carattere '\0' è automaticamente incluso in fondo)
```

Lettura da tastiera:

```
char str[4];
int i;
for (i=0; i < 3; i++)
    scanf(“%c”, &str[i]);
    str[3] = ‘\0’

Oppure

char str[4];
scanf(“%s”, str);
```

Link:

Fondamenti di Informatica/11.04/Array

Upper e Lower bound

Definizione:

In un array, il **lower bound** è l'indice più piccolo dei suoi valori, ed è sempre 0, mentre l'**upper bound** è $N - 1$.

Cambio di Upper e Lower bound:

Mediante l'uso di puntatori è possibile costruire vettori che abbiamo punti di partenza e di arrivo diversi.
Mettiamo per esempio di voler creare un vettore il cui **lower bound** sia 1:

```c
int main() {
    int A[4] = { 1, 2, 3, 4};
    int* p = A - 1;
    printf("%d", p[1]);
}
```

⚠ **È sempre rischioso cambiare le convenzioni!**

Link:
Fondamenti di Informatica/11.04/Array

Definire Tipi Strutturati

Di cosa parliamo:

Fino ad adesso se volevamo definire un array o una struttura dovevamo scrivere:

```
char parola[20];
struct{
int x;
int y;
}vettore;
```

Grazie all'uso di `typedef` possiamo ora scrivere:

```
typedef char parola[20];
parola p1, p2; //due stringhe con 20 caratteri
typedef struct{
int x;
int y;
}vettore;
vettore v1, v2; //due strutture vettore
```

Link:
Tipi definiti dall'utente

Equivalenza nominale

Definizione:

Secondo **l'equivalenza nominale**, due tipi sono definiti **equivalenti** se definiti **nella stessa definizione** oppure se il nome dell'uno **è definito espressamente come identico dell'altro**.

Non adottato nel C, ma nel Pascal ad esempio.

Es:

```
type MioIntero = integer;
type NuovoIntero = integer;
var A: MioIntero;
var B: NuovoIntero;
```

I due tipi `MioIntero` e `NuovoIntero` **non sono equivalenti** perché definiti in una diversa definizione (`A:=B` non è consentito).

Link:
Tipi di Dato

Equivalenza strutturale

Definizione:

In questo tipo di equivalenza (adottata nel C), due tipi sono ritenuti equivalenti se **strutturalmente identici**.

Esempio:

```
typedef int MioIntero;
typedef int NuovoIntero;
MioIntero A;
NuovoIntero B;
```

I due tipi `MioIntero` e `NuovoIntero` sono equivalenti perché **strutturalmente identici** (entrambi `int` per la macchina C).

Link:
Tipi di Dato

Strutture

Definizione:

Una struttura è una collezione finita di varabili **non necessariamente dello stesso tipo**, ognuna identificata da un nome.

Definizione formale:

```
struct [<etichetta>]{
{ <definizione-di-variabile> }
} <nomeVariabile> ;
```

Esempio:

```
struct persona {
    char nome[20];
    int eta;
    float stipendio;
} pers ;
```

Per accedere ai campi si usa **la notazione puntata**.

```
pers.nome, pers.eta, pers.stipendio
```

> ✎ **L'etichetta è opzionale e serve solo per definire con più praticità altre strutture con lo stesso corpo.**

Le Strutture in Memoria:

In memoria i campi di una struttura sono **allocati in modo contiguo** e nello stesso ordine in cui sono stati dichiarati.
L'indirizzo di memoria di una variabile di tipo `struct` coincide con l'indirizzo di memoria **del suo primo membro**.

Differenza con gli Array:

A differenza di quanto accade con gli `array`, il nome della struttura rappresenta **la struttura nel suo complesso**.

Questo dunque **permette di**:

- **Assegnare una struttura ad un'altra** (copia)
- Che una funzione **restituisca una struttura**

- Passare una struttura come parametro a una funzione (passaggio di una copia!)

Tipi definiti dall'utente

Di cosa si tratta:

Il linguaggio C consente di:

- **Ridefinire tipi già esistenti**.
- **Definire dei nuovi tipi strutturati**.
- **Definire dei nuovi tipi enumerativi.**

Link:
Tipi di Dato

Tipi Enumerativi

Definizione:

Un tipo enumerativo è un tipo di dato che consiste in un **insieme ristretto di valori**.
Ogni elemento appartenente al tipo ha un proprio nome (identificatore).
I nomi associati agli elementi del tipo sono trattati come costanti, chiamate costanti enumerative.
I tipi enumerativi sono compatibili con un dato intero.

> ✎ **La loro funzione è semplicemente quella di rendere più leggibile il codice.**

Schema generale:

```
typedef enum {
    a1, a2, a3, ... , aN
} EnumType;
```

Esempio di **implementazione dei valori Booleani**:

```
#include <stdio.h>

typedef enum {
    false,
    true,
}Boolean;

Boolean controllaEta(int eta) {
    Boolean check;
    if (eta >= 18) {
        check = true;
    }
    else {
        check = false;
    }
    return check;
}

int main() {
    int eta;
```

```
    printf("Inserisci la tua age: ");
    scanf_s("%d", &eta);
    if (controllaEta(eta) == false) {
        printf("\nSei ancora minorenne!!!!");
    }
    else {
        printf("\nSei maggiorenne, complimenti.");
    }
    printf("\n");
}
```

Link:

Tipi di Dato

Tipi Ridefiniti

Definizione:

Un nuovo identificatore di tipo viene dichiarato **identico a un tipo già esistente**.

Schema:

```
typedef TipoEsistente NuovoTipo;
```

Es:

```
typedef int MioIntero;
MioIntero X,Y,Z;
int W;
```

Link:

Tipi definiti dall'utente

Matrici

Definizione:

Variabili di tipo `array` con più di una dimensione.

```
<tipo> <identificatore> [dim1][dim2]...[dimn]
```

In particolare vengono chiamate **matrici** gli array con **due dimensioni**.

Per accedere all'elemento che si trova nella riga *i*, colonna *j* si usa la notazione:

```
M[i][j]
//Se con più dimensioni:
M[i][j][k]
```

Memorizzazione:

Le matrici multidimensionali sono memorizzate per righe **in celle contigue**.
L'offset di una cella di memoria di un elemento della matrice indica **quanto quest'ultima disti dalla cella del primo elemento della matrice**.
In una matrice bidimensionale si calcola:

$$LungRiga * i + j$$

Passaggio come parametri:

Nel caso di passaggio come parametro di un array bidimensionale a una funzione, nel prototipo della funzione **va dichiarato necessariamente il numero delle colonne (ovvero la dimensione della riga)**.

Per esempio:
se si vuole passare alla funzione `f()` la matrice `par` occorre scrivere all'atto della definizione della funzione:

```
f(float par[20][30],...)
//oppure
f(float[][30])
```

> **In generale, soltanto la prima dimensione di un vettore multidimensionale può non essere specificata.**

Puntatori:

Non essendoci vincoli sul tipo degli elementi di un vettore, possiamo avere anche

vettori di puntatori.
Per esempio:

```
char* stringhe[4];
```

Il punto cruciale è che questa scrittura mi definisce un vettore di 4 puntatori, questo permette di poter realizzare **righe di lunghezza variabile**.

Link:
Fondamenti di Informatica/11.04/Array

Modello Input-Output

Il modello:

In C l'input e l'output sono gestiti dalla libreria standard `stdio`.
Le due funzioni madri dalle quali parte tutto sono `putchar(ch)` e `ch = getchar`.
Entrambe le funzioni leggono/scrivono un carattere **convertito in** `int`.

> **Ogni altro tipo di I/O può essere costruito sulle due primitive putchar() e getchar().**

printf:

Scrive sul canale di output una serie di valori, effettuando le conversioni richieste ove necessario.
Restituisce il numero di caratteri emessi.
In caso di errore restituisce `EOF`.

```
int printf(char frm[], e1..., ..., eN)
```

La funzione scrive sul canale di output i risultati delle espressioni `e1,...,eN`.

scanf:

```
int scanf(char frm[], add1,…, addN)
```

La funzione legge dal canale di input tanti campi quanti ne specifica la stringa di formato `frm[]`, e li pone in memoria agli indirizzi denotati da `add1, …, addN`.
Restituisce il numero di **campi letti e correttamente convertiti** (0 se non ha letto nulla), o `EOF` in caso di errore.

```
//formati
%c per carattere singolo
%s per stringa di caratteri
```

> ⚠ **scanf() considera finita la stringa al primo spazio o spearatore che incontra! Quindi non si può usare scanf() per leggere una stringa contente spazi.**

Precisazione:

Se scrivo:

```c
scanf("%d%d", &i, &u);
```

`scanf` rimuove **automaticamente gli spazi tra i valori inseriti**.

Se invece scrivo:

```c
scanf("%d %d", &i, &u);
```

Si è costretti ad inserire almeno uno spazio tra i valori.
Dunque bisogna stare attenti!

Link:
C

Stream e File

Cosa sono:

I File e gli Stream sono un livello di astrazione tra il programma e il dispositivo fisico di I/O.

In particolare il **File** è l'astrazione dell'unità fisica o logica in cui i dati sono memorizzati (ad es. disco rigido o tastiera).

Lo **Stream** è invece il canale astratto che il programma usa per leggere o scrivere dati su un File.

In C sono gestiti dalla libreria `<stdio.h>`

File:

Due tipi di File:

- File binari (sequenze ordinate di byte)
- File di testo (sequenze ordinate di caratteri organizzate in righe)

Aprire un File in C:

Si usa la funzione:

```
FILE* fopen(char fname[], char modo[])
```

Questa funzione apre il file di nome `fname` nel `modo` specificato e restituisce un puntatore a `FILE`.

`modo[]` può essere:

- `r` apertura in lettura (read)
- `w` apertura in scrittura (write)
- `a` apertura in aggiunta (append) --> non si cancella nulla di ciò che è stato scritto e il nuovo contenuto verrà aggiunto in fondo al file.
  Seguito opzionalmente da:
- `t` apertura in modalità testo (default)
- `b` apertura in modalità binaria
  Ed eventualmente:
- `+` apertura con possibilità di modifica

Questa funzione restituisce `NULL` in caso l'operazione sia fallita --> unico modo di verificare il corretto caricamento del file.

Chiudere un File in C:

Si usa la funzione:

```
int fclose(FILE*)
```

Restituisce:

- `0` se l'operazione è andata a buon fine
- `EOF` in caso di errore

Controllare la Fine del File:

La fine del File viene rilevata in base all'esito delle operazioni di lettura: se il valore restituito dalla funzione è `EOF` vuol dire che il file **è terminato**, se precedentemente **letto correttamente**.

File di Testo:
I canali di I/O standard sono dei File di testo già aperti:

- `stdin` è un file di testo aperto in lettura, di norma **agganciato alla tastiera**.
- `stdout` è un file di testo aperto in scrittura, di norma **agganciato al monitor**.
- `stderr` è un altro file di testo aperto in scrittura, di norma **agganciato al monitor**.

Stream predefiniti:
Ci sono tre canali predefiniti:
`stdin, stdout, stderr`
Sono gli unici Stream che non devono essere chiusi alla fine del programma.

> **La lettura e scrittura su File sono operazioni sequenziali! Come se fosse la testina di una macchina da scrivere che procede nel file in modo ordinato, dall'alto verso il basso. Se si vuole riportare la testina in cima al file si usa la funzione `rewind(FILE*f)`**

Esempio lettura da file testo:

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include "element.h"
#include "list.h"

int main() {
    // 1. Puntatore al file e apertura
```

```c
    FILE *fp = fopen("dati.txt", "r");
    if (fp == NULL) {
        printf("Errore: impossibile aprire il file.\n");
        return -1;
    }

    list miaLista = emptyList(); // Inizializzo la lista vuota
    element e;                   // Variabile temporanea per la lettura

    // 2. Ciclo di lettura
    // fscanf restituisce il numero di campi letti correttamente
    while (fscanf(fp, "%s %d %d", e.nome, &e.info1, &e.info2) == 3) {
        // 3. Inserimento nella lista
        miaLista = cons(e, miaLista);
    }

    // 4. Chiusura file e visualizzazione
    fclose(fp);
    printf("Dati letti dal file:\n");
    showList(miaLista);

    // 5. Pulizia memoria (Fondamentale!)
    freeList(miaLista);

    return 0;
}
```

Link:
Modello Input-Output

Aggiunta File

Definizione:

- È **un’astrazione fornita dal sistema operativo**, per consentire la memorizzazione di informazioni su memoria di massa
- È un’astrazione di memorizzazione di dimensione **potenzialmente illimitata**, tipicamente **ad accesso sequenziale** (esiste il concetto di posizione corrente).

File Binari

Uso:

Un **file binario** è una **sequenza di byte**.

Può essere usato per archiviare su memoria di massa **qualunque tipo di informazione**.

Input e output avvengono sotto forma di una **sequenza di byte**.

La fine del file è sempre rilevata in base all'**esito delle operazioni di lettura!**
Infatti:

- Non ci può essere un carattere speciale per indicarne la fine (un file binario non è un sequenza di caratteri).
- Qualsiasi byte si scegliesse come marcatore, potrebbe sempre capitare nella sequenza.

Lettura e Scrittura:

Scrittura:

```
int fwrite(addr, int dim, int n, FILE* f);
```

- Scrive sul file `n` elementi, ognuno grande `dim` byte (complessivamente scrive dunque `d*dim` byte)
- Gli elementi da scrivere vengono prelevati dalla memoria a partire dall'indirizzo `addr`
- Restituisce il numero di elementi (non di byte) effettivamente scritti, che possono essere meno di `n`

Lettura:

```
int fread(addr, int dim, int n, FILE* f);
```

- Legge dal file `n` elementi, ognuno grande `dim` byte (complessivamente, tenta di leggere quindi `n*dim` byte)
- Gli elementi da leggere vengono scritti in memoria a partire dall'indirizzo `addr`
- Restituisce il **numero di elementi** (non di byte) effettivamente letti, che possono essere meno di `n` **se il file finisce prima**

Controllare il valore restituito è il solo modo per sapere che cosa è stato letto, e in particolare **per scoprire se il file è terminato**.

Rappresentazione Interna ed Esterna di un numero

Definizione:

Rappresentazione interna:

La rappresentazione interna è il modo in cui il computer memorizza, elabora ed esegue calcoli sui dati. **Il formato è sempre binario**.

Rappresentazione esterna:

La rappresentazione esterna è il modo in cui i dati vengono visualizzati dall'utente. Il formato è quello che può essere letto dall'uomo, **tipicamente sequenze di caratteri**.

Nei File:
Nella gestione dei dati immagazzinati in file, questa differenza è cruciale.

Output di numeri:

Osserviamo la differenza tra:

```
x = 314
fprintf(file, "%d", x);
fwrite(&x, sizeof(int), 1, file);
```

`fprintf` la possiamo usare solo nei **file testuali**, perché scrive sul file la sequenza di caratteri ASCII, cioè la **rappresentazione esterna del carattere**, ossia i 3 byte `00110011 00110001 00110100`

Invece `fwrite` la possiamo usare solo nei **file binari**, perché scrive sul file direttamente la sequenza di numeri binari che rappresentano quel valore, ovvero i byte `01111010 11101010`

Input di numeri:

Osserviamo la differenza tra:

```
x = 314
fscanf(file, "%d", &x);
fread(&x, sizeof(int), 1, file)
```

`fscanf` preleva la stringa di caratteri ASCII che costituisce la rappresentazione esterna del numero e la **converta nella rappresentazione interna**.

`fread` invece preleva i dati in rappresentazione interna **senza fare alcuna conversione**.

Size_t

Definizione:

`size_t` è un tipo di dato in C non primitivo (definito con `typedef` in `<stdio.h>`) per un **intero senza segno**.
Si usa come **valore di ritorno della quantità di memoria occupata** da un certo dato.
Per esempio è il valore di ritorno di:

```
sizeof();
fread();
fwrite();
```

Per stampare un valore `size_t`:

```
printf("%zu", //valore di tipo size_t)
```

Algoritmi di ordinamento

Definizione:

Se abbiamo un array, e vogliamo ordinare i suoi elementi in ordine crescente o decrescente, abbiamo principalmente **cinque modi**:

- **Naïve sort** (semplice, intuitivo, poco efficiente)
- **Bubble sort** (semplice, un po' più efficiente)
- **Insert sort** (intuitivo, abbastanza efficiente)
- **Merge sort** (non intuitivo, molto efficiente)
- **Quick sort** (non intuitivo, alquanto efficiente)

Naïve sort:

Procedimento:

- Un ciclo esterno, basato sul contatore `i`, pone come valore minimo dell'array il valore di indice `i`. Quello che però è fondamentale non è tanto il valore in sé, quanto l'indice. Dunque: `min = i`
- Un ciclo interno, basato sul contatore `j`, controlla che ogni singolo elemento successivo ad `i` sia effettivamente maggiore del minimo `v[i]`
- Se il minimo risulta maggiore di un valore, allora l'indice minimo diventa `j`
- Infine, fuori dal ciclo interno, se l'indice minimo `min` è diverso dall'indice `i`, allora si scambiano i valori tra `v[i]` e `v[min]`. In questo modo sappiamo che il valore `v[i]` sarà sempre il più piccolo tra i valori che vengono dopo.

Complessità:

La complessità dell'algoritmo è $O(n^2/2)$, questo vuol dire che il tempo necessario all'algoritmo per svolgere la sua funzione **è proporzionale al quadrato della dimensione dell'input**.
L'inefficienza di questo algoritmo è data dal fatto che indipendentemente dall'input i conti **saranno sempre gli stessi** (a parità di dimensione), anche se l'array è già ordinato!

Implementazione:

```c
#include <stdio.h>

void selectionSort(int v[], int n) {
    int i, tmp, min, j;
    for (i = 0; i < n - 1; i++) {
```

```
            min = i;
            for (j = i + 1; j < n; j++) {
                if (v[min] > v[j]) {
                    min = j;
                }
            }
            if (min != i) {
                tmp = v[i];
                v[i] = v[min];
                v[min] = tmp;
            }
        }
    }
```

Bubble sort:

Procedimento:

- La novità è l'implementazione della variabile `ordinato`, che permette all'algoritmo di rilevare se l'array è già ordinato.
- Implementiamo un ciclo esterno dipendente dalla dimensione dell'array `n` e dalla variabile `ordinato`. Fino a quando `n>1` e `ordinato == 0` il ciclo continuerà, decrementando `n` ad ogni iterazione. All'inizio del ciclo impostiamo la variabile `ordinato` uguale ad `1`. Se il suo valore non verrà cambiato, il ciclo terminerà.
- Implementiamo ora un ciclo interno, dipendente dal contatore `i`. In questo ciclo si andrà a controllare che il valore di `v[i]` non sia maggiore di `v[i+1]`. Se ciò accade, si scambiano le due variabili e si imposta `ordinato = 0`
- Ad ogni "passata" l'algoritmo porta il valore più grande all'ultimo posto dell'array, e riduce la sua analisi all'array di dimensione `n-1`. Se non avvengono scambi, vuol dire che l'array è ordinato e l'algoritmo termina.

Complessità:

Grazie al fatto che l'algoritmo è in grado di fermarsi non appena rileva una parte dell'array già ordinato, l'algoritmo risulta più efficiente del naïve sort.
Infatti:

- Nel caso peggiore la sua complessità sarà $O(n^2/2)$ come nel caso precedente.
- Nel caso migliore però la sua complessità sarà $O(n)$!

Implementazione:

```c
void bubbleSort(int v[], int n) {
    int i, tmp, ordinato = 0;
    while (n > 1 && ordinato == 0) {
        ordinato = 1;
        for (i = 0; i < (n - 1); i++) {
            if (v[i] > v[i+1]) {
                tmp = v[i];
                v[i] = v[i+1];
                v[i+1] = tmp;
                ordinato = 0;
            }
        }
        n--;
    }
}
```

Insert sort:

Procedimento:

- Si parte dando per scontato che il primo elemento dell'array sia ordinato
- Implementiamo ora un ciclo esterno che valuti l'elemento successivo al primo. All'inizio di questo ciclo definiamo la variabile `chiave` uguale a `v[i]`, che è il contatore del ciclo, che però parte da `1`. `chiave` indica il **primo valore dell'array non ordinato**.
- Ora definiamo `j` uguale a `i-1`. Questa variabile rappresenta invece **l'ultimo elemento dell'array ordinato**.
- A questo punto implementiamo un ciclo interno, che valuta gli elementi dell'array all'indietro. Sostanzialmente andiamo a valutare se `v[j]` è maggiore di `chiave`. Quando scopriamo esserci un valore maggiore a chiave, stabiliamo che la posizione successiva a quel valore (`v[j+1]`) deve essere uguale al valore stesso `v[j]`. In questo modo stiamo solo andando a creare uno spazio che riempiremo appena fuori dal ciclo.
- Usciti dal ciclo interno, impostiamo `v[j+1]` uguale a `chiave`.
- Attenzione perché nell'ultimo ciclo interno `j` è decrementata di uno, dunque stiamo andando a sostituire l'elemento che nel ciclo era `v[j]`.
- Se non è stata necessaria alcuna sostituzione, `j+1` sarà l'ultima posizione del nuovo array ordinato.

Complessità:

- Nel caso peggiore, necessita di $O(n^2)$ valutazioni e spostamenti.
- Nel caso migliore $N-1$ valutazioni, senza alcun spostamento.
- Nel caso medio circa $O(N^2/4)$ valutazioni e spostamenti

Implementazione:

```
void insertSort(int v[], int n) {
    int chiave, i, j;
    for (i = 1; i < n; i++) {
        chiave = v[i];
        j = i - 1;
        while (j >= 0 && v[j] > chiave) {
            v[j + 1] = v[j];
            j--;
        }
        v[j + 1] = chiave;
    }
}
```

Merge sort:

Procedimento:

- Introduciamo per prima la funzione `mergeSort`. Questa funzione sarà quella che compirà il passo ricorsivo e che dividerà il nostro array in array lunghi la metà di quello originario, fino ad arrivare ad array con un solo elemento, e dunque per definizione già ordinato.
- Nella `mergeSort` andiamo poi a chiamare la funzione `merge`, che come scopo ha quello di ricostruire tutti i piccoli array ordinati in un unico array ordinato.
- In partica, la funzione `merge` prende l'array e lo valuta diviso in due parti: la parte destra e la parte sinistra. A questo punto confronta il primo elemento della parte sinistra con il primo elemento della parte destra, e se quest'ultimo è maggiore del primo, viene posizionato nella posizione `i` dell'array provvisorio.
- Questo processo viene fatto finché una delle due parti non finisce gli elementi, e a questo punto gli elementi rimanenti dell'altra parte (già ordinati grazie alla ricorsione) vengono affiancati successivamente nell'array.
- Successivamente si trasferiscono i valori dell'array provvisorio a quelli dell'array originale.

- Sì, è da imprecazione tutto ciò.

Complessità:

- Il Merge Sort ha sempre complessità $O(N \log_2 N)$
- Si dimostra che $O(N \log_2 N)$ è un limite inferiore alla complessità del problema dell'ordinamento di un array e dunque nessun algoritmo, presente o futuro, potrà far meglio di $O(Nlog_2N)$

Implementazione:

```
void merge(int array[], int l, int m, int r, int vout[]) {
    int i = l;
    int j = m + 1;
    int k = l;
    //Ciclo di controllo
    while (i <= m && j <= r) {
        if (array[i] <= array[j]) {
            vout[k] = array[i];
            i++;
        }
        else {
            vout[k] = array[j];
            j++;
        }
        k++;
    }
    //Affiancamento
    while (i <= m) {
        vout[k] = array[i];
        i++;
        k++;
    }
    while (j <= r) {
        vout[k] = array[j];
        j++;
        k++;
    }
    //Assegnazione array temp ad array
    for (i = l; i <= r; i++) {
```

```
        array[i] = vout[i];
    }
}

void mergeSort(int array[], int l, int r, int vout[]) {
    if (l < r) {
        int m = l + ((r - l) / 2);
        //Parte sinistra
        mergeSort(array, l, m, vout);
        //Parte destra
        mergeSort(array, m + 1, r, vout);
        merge(array, l, m, r, vout);

    }
}
```

Quick sort:

Procedimento:

- Creiamo per comodità una funzione `swap` che permetta all'occorrenza di invertire i valori di due variabili.
- Creiamo la funzione `partition` che faccia questo procedimento: prende in ingresso un array, e assegna alla variabile `pivot` il valore del suo ultimo numero. A questo punto si introducono due contatori, `i` che è uguale all'indice più basso dell'array meno 1, e `j`, che invece è uguale all'indice più basso.
- Si sviluppa dunque un ciclo che continua fino a quando `j` è minore o uguale al valore massimo meno 1. In questo scriviamo un'istruzione condizionale: se il valore di `array[j]` è minore del valore di `pivot`, allora si incrementa `i` di 1 e si scambia il valore di `array[i]` con quello di `array[j]`. Si compie questo processo per tutti i valori dell'array fino ad arrivare a quello prima del pivot.
- A questo punto usciamo dal ciclo e scambiamo l'ultimo valore dell'array, che era il nostro pivot, con il valore dell'array in posizione `i+1`. La funzione deve restituire come ritorno l'indice `i+1`
- Ora dobbiamo sviluppare la parte chiave dell'algoritmo. La funzione `quickSort` compie come prima cosa un controllo: se l'array passatogli contiene un solo elemento o è vuoto allora la funzione termina. Altrimenti dichiara la variabile `pivot` e gli assegna il valore di ritorno di `partition`, e qui avviene la chiamata ricorsiva.

- La funzione infatti chiama se stessa due volte: la prima con la parte dell'array che va dal valore iniziale al valore precedente al pivot, la seconda che va dal valore dopo il pivot al valore finale.
- Fine, questo forse il più bello. Bel equilibrio tra comprensibilità ed efficienza.

Complessità:

La complessità **dipende da come viene scelto il pivot**:

- Se infatti viene scelto "male" (uno dei due sub-array è 0), la complessità è intorno a $O(N^2)$, cioè paragonabile al Bubble sort.
- Se però il pivot è scelto in modo da avere due sub-array con la stessa dimensione, allora la complessità raggiunge quella del merge-sort:$O(N \log_2 N)$.

Implementazione:

```
void swap(int* a, int* b) {
    int tmp = *a;
    *a = *b;
    *b = tmp;
}
int partition(int array[], int low, int high) {
    int pivot = array[high];
    int i = (low - 1);
    int j = low;
    while (j <= (high - 1)) {
        if (array[j] <= pivot) {
            i++;
            swap(&array[j], &array[i]);
        }
        j++;
    }
    swap(&array[high], &array[i + 1]);
    return (i + 1);
}

void quickSort(int array[], int low, int high) {
    if (low < high) {
        int pivot = partition(array, low, high);
        quickSort(array, low, pivot - 1);
```

```
        quickSort(array, pivot + 1, high);
    }
}
```

Algoritmi

Implementazione **MergeSort**:

```
int n = length(elenco);
element* array = (element*) malloc(n * sizeof(element));
element* vout = (element*) malloc(n * sizeof(element)); // Array di
supporto

// ... copia i dati da lista ad array ...

mergeSort(array, 0, n - 1, vout);

free(array);
free(vout);
```

Implementazione **QuickSort**:

```
#include "Algoritmi.h"
#include "list.h"

void esempioOrdinamento(list elenco) {
    int n = length(elenco);
    if (n <= 0) return;

    // 1. Alloca l'array dinamico per contenere gli elementi
    element* array = (element*) malloc(n * sizeof(element));

    // 2. Trasferisci i dati dalla lista all'array
    list temp = elenco;
    for (int i = 0; i < n; i++) {
        array[i] = head(temp);
        temp = tail(temp);
    }
```

```
    // 3. Chiama il Quick Sort (usando gli indici 0 e n-1)
    quickSort(array, 0, n - 1);

    // 4. Stampa il risultato o fai quello che serve
    for (int i = 0; i < n; i++) {
        printElement(array[i]);
    }

    // 5. Libera la memoria dell'array
    free(array);
}
```

Classi di memoria in C

4 tipi:

Il linguaggio C prevede 4 classi di memoria da preporre alle variabili:

- `auto`
- `static`
- `extern`
- `register`

Auto:

- Classe di memoria di **default** per le variabili locali
- Vengono create all'**attivazione della funzione** (record di attivazione)
- Non vengono inizializzate automaticamente
- Quando il servitore restituisce il controllo al cliente queste variabili **vengono distrutte** e perdono ogni valore assegnato loro.

Static:

- Non sono mai previste di default e quindi la parola a chiave `static` **deve essere esplicitata** per introdurle.
- L'allocazione e l'inizializzazione avviene **all'inizio del programma** nelle aree di memoria dei dati globali del programma.
- Per default le variabili statiche **sono sempre inizializzate a zero**.
- Una variabile statica **conserva il proprio valore tra una chiamata e l'altra** della funzione in cui è definita.

Extern:

- Lo sono per default le variabili globali e sopravvivono **per tutta l'esecuzione del programma**
- La parola chiave `extern` deve essere esplicitata per accedere a variabili globali **definite in altri file**
- Una variabile locale esterna **non è quindi memorizzata nel record di attivazione** della funzione.

Register:

- Non sono mai previste di default e quindi la parola chiave `register` deve essere esplicitata per introdurle.
- Rappresenta il modo per allocare variabili automatiche **su registri ad accesso**

più veloce e che compiono operazioni **più rapidamente**

- Su tali variabili **non si può utilizzare l'operatore di estrazione di indirizzo** `&`
- Un uso tipico di questa classe è per gli indici di ciclo:

```
void f(void){
    register int i;
    for (i=0; i<SIZE;i++)
    {...}
}
```

Variabili Globali

Definizione:

- Una variabile globale è una variabile **definita fuori da qualunque funzione**.
- Il suo tempo di vita è **l'intero programma**.
- Il suo scope (regione del codice in cui la variabile è accessibile) è il file in cui è dichiarata **dal punto in cui è scritta in avanti**.

Dichiarazione e Definizione:

Come distinguere la **dichiarazione di una variabile globale** dalla sua **definizione**?

Si usa la parola chiave `extern`:

```
int trentadue = 10; //definizione (con inizializzazione)
extern int trentadue //dichiarazione
```

Se una variabile globale viene dichiarata, deve essere **definita in un'altra parte del codice** o in un **altro file** appartenente al progetto.

Principalmente le variabili globali servono a **due scopi**:

Scambio di informazioni:

Scambiare informazioni **fra cliente e servitore** in modo alternativo al passaggio dei parametri.

In questo caso le variabili globali sono un mezzo **bidirezionale**: la funzione può sfruttarle per memorizzare un'informazione **destinata a sopravviverle**.

La funzione è però così dipendente dall'ambiente esterno e dunque **limitata nella sua riusabilità**.

Costruzione di specifici componenti software:

Permettono infatti di creare speciali tipi di funzioni basate **su uno stato**.
Per esempio una funzione che ogni volta che viene invocata restituisca il successivo numero dispari necessita di **un valore al di fuori della funzione che incrementi**.
Dunque non è una funzione in senso matematico, perché ogni volta che viene interrogata **il valore è diverso**.

Variabili Statiche

Definizione:

In C, una variabile può essere dichiarata `static`:

- È **permanente** come tempo di vita
- È protetta, in quanto è visibile solo **entro il suo scope di definizione**.

> **Nel caso di una variabile globale `static`, ogni tentativo di accedervi da altri file, tramite dichiarazioni `extern`, sarà impedito.**

Variabili statiche dentro a funzioni:

Le variabili statiche si possono anche definire **dentro ad una funzione**.
In tal modo:

- **Sono protette**, cioè visibili solo dentro alla funzione.
- Sono anche **permanenti**, in quanto il loro tempo di vita diventa quello dell'intero programma.
  Dunque le variabili globali permettono di **costruire componenti dotati di stato**, ma **indipendenti dall'esterno**.

I due effetti:

Quindi, la parola chiave `static` ha sempre e comunque due effetti:

- Rende l'oggetto **permanente**
- Rende l'oggetto **protetto** (invisibile fuori dal suo scope di definizione)
  Ma se ne vede sempre **uno solo per volta**
- Una variabile definita in una funzione, che è comunque **protetta**, **viene resa permanente**
- Una variabile globale, già di per sé **permanente**, **viene resa protetta**

Allocazione della Memoria

Definizione:
Con "Allocazione della memoria" si intende l'**assegnazione di un blocco di memoria RAM** ad un programma in esecuzione.

In C ci sono tre modalità di allocazione della memoria:

Allocazione automatica

Riguarda le variabili (non `static`) con scope locale. La memoria viene **automaticamente allocata** sullo stack (record di attivazione) e **automaticamente deallocata** al termine della funzione in cui è dichiarata.

Allocazione statica

Riguarda le **variabili globali** e le **variabili locali dichiarate** `static`.
La memoria viene allocata **prima dell'esecuzione del programma** e rilasciata **al termine**.

Allocazione dinamica

Memoria richiesta a `runtime` esplicitamente dal programmatore (mediante specifiche funzioni di libreria).
Viene allocata dinamicamente nelle memoria `heap` a cui si può accedere **solo mediante puntatori**. Deve anche **essere rilasciata esplicitamente**.

Allocazione Dinamica

In cosa consiste:

Per **chiedere nuova memoria** o **rilasciarla al momento opportuno**, si usano specifiche funzioni di libreria i cui prototipi sono dichiarati nella libreria `stdlib.h` e che mandano la richiesta al sistema operativo.
Queste funzioni sono `malloc` e `free`.

Malloc:

La funzione:

```
void* malloc(size_t dim)
```

Chiede al sistema di allocare **un'area di memoria contigua** grande `dim` byte.

`size_t` rappresenta un tipo specifico già analizzato.

Restituisce l'indirizzo dell'area di memoria allocata (`NULL` se l'allocazione non è stata possibile)

Il sistema operativo preleva la memoria richiesta nell'area `heap`

⚠ **malloc() restituisce un puro indirizzo, ossia un puntatore "senza tipo". Per assegnarlo a uno specifico puntatore occorre un cast esplicito.**

Bisogna prestare attenzione anche perché l'indirizzo restituito da `malloc()` può anche essere assegnato ad un tipo la cui dimensione **non corrisponde al tipo allocato**.

Es:
Per allocare dinamicamente 12 byte:

```
float *p;
p = (float*) malloc(12);
```

Per allocare dinamicamente lo spazio necessario per 5 interi:

```
int *p;
p = (int*) malloc(5*sizeof(int));
```

🔥 **Cruciale è che l'espressione passata a malloc() può contenere variabili, dunque è possibili costruire array dinamici le cui dimensioni vengono**

scelte dal programma in base alle esigenze del momento!

Free:

La deallocazione della memoria avviene tramite la procedura:

```
void free(void*p)
```

È buona norma liberare la memoria per non saturarla.

Una volta passato un puntatore, il sistema sa quanta memoria deallocare per quel puntatore (ricorda la relativa `malloc`).

Se la memoria non viene correttamente deallocata si verifica il **memory leaking** (consumo di memoria).

Tempo di vita:

Il tempo di vita di un'area dati dinamica non è legato a quello delle funzioni. Dunque, un'area dinamica può sopravvivere **anche dopo che la funzione che l'ha creata è terminata**.

Dangling Reference

Definizione:

Si presenta quando un puntatore punta ad aree che sono state deallocate.
Es:

```
int *p;
p=(int*)malloc(5*sizeof(int));
free(p);
p[0] = 13;
p[1] = 18;...
*(p+4) = -20;
```

Dopo `free` il blocco di memoria puntato da `p` viene deallocato, ma il valore del puntatore `p` non cambia. Usare `p` diventa dunque pericoloso perché potrebbe star puntando ad un'area di memoria allocata per altri scopi.

Garbage Collection

Definizione:

Fu inventata nel 1959 per il linguaggio LISP.

È una **gestione automatica della deallocazione della memoria**. Il sistema sa sempre quanti e quali puntatori puntano ad una certa area di memoria, e quando un'area non è più puntata da nessuno **viene recuperata tramite opportuna deallocazione**.

In C non è presente e deve essere **tutto gestito dall'utente** ;;).

Lista

Definizione:

Una **Lista** è un **tipo di dato astratto** (ADT) che cattura il concetto di **sequenza ordinata di elementi**.
Come ogni tipo dato astratto, la lista è definita in termini di:

- Dominio dei suoi elementi D
- Operazioni $\mathfrak{J}$
- Predicati Π

Una lista semplice è un tipo di dato astratto in cui:

- D può essere chiunque
- $\mathfrak{J}$ = { `cons` , `head` , `tail` , `emptyList` }
- Π = { `empty` }

Capiamo le operazioni e i predicati:

$\mathfrak{J}$:

- `cons` (costruzione) aggiunge un nuovo elemento alla lista
- `head` (selezione) restituisce il primo elemento della lista
- `tail` (selezione) restituisce la lista **privata del suo primo elemento**
- `emptyList` (costruzione) costruisce una lista vuota

Π:

- `empty` verifica se la lista è vuota (restituisce vero o falso)

Implementazione:

In C non esista il tipo predefinito `lista`, e dunque deve essere definito a partire da altre strutture dati, in particolare **da vettori o puntatori**.

Implementazione tramite vettori:

Si utilizza un vettore per memorizzare gli elementi della lista uno dopo l'altro (rappresentazione sequenziale).

Vantaggi:

- Implementazione molto semplice
- Accesso agli elementi tramite indice molto rapido

Svantaggi:

- Dimensioni fisse del vettore
- L'inserimento e la cancellazione di un certo numero di elementi risultano molto costosi da un punto di vista di tempo e risorse.

Implementazione tramite liste concatenate:

A ogni elemento si associa un'indice che permette di individuare **la posizione dell'elemento successivo**.
Dunque la sequenzialità degli elementi della lista non è più rappresentata **mediante l'adiacenza delle locazioni di memorizzazione**.
Gli elementi della lista chiamati **nodi**, i riferimenti **archi**.

ESEMPIO
[5, 8, 21]



Vantaggi:

- Struttura altamente dinamica in cui i nodi della lista **non sono necessariamente contigui in memoria**.
- L’inserimento o cancellazione degli elementi non comporta la riorganizzazione dell’intera struttura con lo spostamento di elementi **ma solo l’aggiornamento di puntatori**.

Inconvenienti:

- L’accesso ad un elemento non può avvenire in modo diretto, ma richiede l’attraversamento sequenziale di un certo numero di nodi della lista.

Tipi di dato astratto

Definizione:

Un **tipo di dato astratto** (ADT) definisce una categoria concettuale con le sue proprietà.
È definito da:

- **Un dominio** (l'insieme dei valori con cui tratterà)
- Un insieme di **operazioni ammissibili su oggetti di quel tipo**

La costruzione:

In C, un ADT si costruisce definendo:

- Il nuovo tipo con `typedef`
- Una funzione per ogni operazione

Importante anche la struttura di un ADT, che comprende:

- Un file header, contente il `typedef` e le **dichiarazioni delle funzioni**
- Un file di implementazione, **contente le definizioni delle funzioni**.

Operazioni di un ADT:

Si possono definire 5 operazioni per un ADT:

- **Costruttori** (costruiscono un oggetto di questo tipo, a partire dai suoi "costituenti elementari", ad es concatenare due stringhe per formarne una nuova).
- **Selettori** (leggono una parte dell'oggetto senza modificarlo)
- **Predicati** (verificano la presenza di una proprietà sull'oggetto, restituendo vero o falso)
- **Funzioni** (agiscono in vario modo sugli oggetti)
- **Trasformatori** (modificano lo stato di un oggetto esistente, ad es incrementare un oggetto di un'unità).

I limiti in C:

- Gli ADT così realizzati funzionano, ma molto dipende **dall'autodisciplina del programmatore**.
- Non esiste **alcuna protezione** contro un uso scorretto dell'ADT.

Infatti:

- L’organizzazione suggerisce di operare sull’oggetto **solo tramite le funzioni previste**, ma **non riesce a impedire di aggirarle** a chi lo volesse (ad esempio: `counter c1; c1++;`)
- La struttura interna dell’oggetto **è visibile a tutti** (nella `typedef`).

Esempio:

Esempio di implementazione di un contatore tramite un ADT:

`counter.h:`

```
#pragma once
typedef int counter;

void inc(counter*);

void reset(counter*);
```

`counter.c:`

```
#include "counter.h"

void inc(counter* c) {
    *c+= 1;
}

void reset(counter* c) {
    *c = 0;
}
```

`main.c`

```
#include <stdio.h>
#include "counter.h"

int main() {
    counter c;
    reset(&c);
    inc(&c);
    printf("%d", c);
}
```

Array e come gestirli

Capiamo:

Se in un programma definiamo un array, il nome dell'array sarà sempre un puntatore costante all'indirizzo di memoria del suo primo elemento.
L'unica eccezione è per gli array di caratteri, infatti se inseriamo l'array come parametro in `printf("%s", array)` restituirà l'intera stringa.

> ⚠ **Questo è dovuto al comportamento di printf che continua a leggere caratteri fino a quando non incontra il carattere "\0".**

Comparare :

Siccome il nome di un array punta sempre ad un indirizzo di memoria, non ha senso comparare una stringa e un array così:

```
char Array[] = "fine";
if (Array == "fine"){};
```

Stiamo comparando un indirizzo di memoria (in particolare un tipo `void*`) con una stringa.
Il metodo corretto è usare:

```
#include <string.h>

if (strcmp(Stringa1, Stringa2) != 0){
    printf("Le due stringhe sono diverse!");
}
```

Buone Abitudini 

Ohhh! Iniziamo finalmente con la listona:

Inizializzare TUTTO:

> **Per evitare errori, warnings e accidenti vari, inizializza sempre i valori degli identificatori all'inizio delle funzioni.**

```c
int main(){
    int x = 0;
    int A[] = {0};
    char B[] = {0};
    ecc...
}
```

Controllare l'output di scanf:

> **Quando usiamo `scanf`, è sempre bene controllare che la lettura sia andata a buon fine. Sfruttiamo il fatto che `scanf` restituisce il numero degli elementi immagazzinati.**

```c
if (scanf("%s", &a) != 1) {
    printf("Errore di lettura");
    return 1;
}
```

In questo caso `scanf` deve leggere solo 1 stringa, se così non fosse allora non restituirebbe 1 (0 o `EOF`).

Uso consapevole di `printf`:

> **Il primo consiglio consiste nell'inserire sempre `\n` alla fine di un una stampa.**

```c
printf("Hello World!\n");
```

> **Abbi in mente la sintassi di printf**

```
%[flag][larghezza][.precisione][lunghezza]tipo$$
```

Flag:

Flag	Descrizione	Esempio
`-`	**Allineamento a sinistra** (default è a destra).	`%-10s`
`+`	Fornisce sempre il **segno** (`+` o `-`) per i numeri.	`%+d`
`0`	Riempie con **zeri iniziali** anziché spazi.	`%05d` (stampa `00123`)
`#`	Usa una forma alternativa (es. `0x` per esadecimale).	`%#x`

Larghezza:

Specifica la **larghezza minima** del campo di output. Se il valore stampato è più corto, viene riempito con spazi (o zeri, se è presente il flag `0`).

Precisione :

La precisione è preceduta da un punto (`.`) e ha diversi significati a seconda del tipo:

Tipo	Descrizione	Esempio
Float	Numero di cifre dopo la **virgola decimale**.	`printf("%.2f", 3.14159);` (stampa `3.14`)
Interi	Numero minimo di cifre che devono apparire.	`printf("%.5d", 123);` (stampa `00123`)
Stringhe	**Numero massimo** di caratteri da stampare.	`printf("%.3s", "Ciao");` (stampa `Cia`)

Modificatori di lunghezza:

Modifica la dimensione del tipo di argomento (es. `long int` anziché `int`).

Modificatore	Descrizione	Specificatore di Formato	Argomento Atteso	Esempio d'Uso
`l`	Per `long int` o `double` (`long double` richiede `%Lf`).	`%ld` , `%lu` , `%lf`	`long int` , `unsigned long int` , `double`	`printf("%ld", grande_int);`

Modificatore	Descrizione	Specificatore di Formato	Argomento Atteso	Esempio d'Uso
`ll`	Per `long long int` o `unsigned long long int`.	`%lld`, `%llu`	`long long int`, `unsigned long long int`	`printf("%lld", molto_grande);`
`h`	Per `short int` o `unsigned short int`.	`%hd`, `%hu`	`short int`, `unsigned short int`	`printf("%hd", corto);`
`z`	Per il tipo `size_t` (usato per dimensioni e indici).	`%zu`	`size_t`	`printf("%zu", dimensione);`
`L`	Per il tipo `long double` (solo con `%f`, `%e`, `%g`).	`%Lf`, `%Le`, `%Lg`	`long double`	`printf("%Lf", numero_lungo);`

Mente Algoritmica 🧠

Intro:

Per formare la nostra mente al meglio e diventare dei bravi **problem solver** (anche chiamati ingegneri) dobbiamo passare in mezzo **a 5 fasi** quando ci troviamo davanti ad un problema.

Fase 1:

Comprendere e formalizzare il problema:

1. **Definisci Input e Output**: quali dati riceve il tuo programma? Cosa deve produrre in uscita?
2. **Identifica i Vincoli**: ci sono limiti sul tipo di dati o sulle operazioni? (ad es i numeri devono essere solo interi, la lista non deve mai essere vuota ecc...)
3. **Esempi a mano**: crea un semplice esempio e risolvilo passo dopo passo tu stesso. Questo aiuta a definire la logica senza il disturbo della sintassi del linguaggio.

Fase 2:

Divide et impera:

1. **Identifica i Moduli**: nessun problema complesso è risolvibile in un unico blocco. Spezzalo in problemi più semplici, ognuno dei quali deve diventare una funzione o un modulo separato.
2. **Concentrati su Uno alla Volta**: risolvi e testa ogni funzione singolarmente.

Fase 3:

Lo Pseudocodice:

1. **Definire la logica**: non scrivere il codice subito in un linguaggio informatico! Usa piuttosto lo pseudocodice! Descrivi l'algoritmo con una sintassi chiara, universale e concisa. In questo modo puoi focalizzarti sulla pura logica e non sulla sintassi. Es:

```
INIZIO
SE lista_numeri è vuota ALLORA
    RESTITUISCI ERRORE
ALTRIMENTI
    massimo = primoElemento di lista_numeri
    PER ogni numero in lista_numeri A PARTIRE dal secondo FAI
        SE numero > massimo ALLORA
            massimo = numero
```

```
        FINE SE
    FINE PER
    RESTITUISCI massimo
FINE SE
FINE
```

Fase 4:

Traduci e Codifica (focus sulla sintassi):

1. **Traduzione diretta**: converti lo pseudocodice nella sintassi del tuo linguaggio.
2. **Commenti**: commenta il codice in modo da renderlo più comprensibile nei passaggi meno intuitivi.
3. **Stile**: ricordati, siamo artisti del codice. Sii ordinato, pulito e chiaro.

Fase 5:

Test e Debugging:

1. **Casi semplici**: esegui il programma con esempi semplici che hai risolto a mano nella fase 1.
2. **Casi limite**: metti alla prova il codice con gli estremi che spesso nascondono bug logici (liste vuote, valori massimi e minimi...)
3. **Usa il Debugger**: se ci sono errori o problemi, usa il debugger per tenere traccia del flusso logico.

Complemento a due

Conversione:

Vogliamo convertire dei numeri decimali in numeri binari usando il complemento a due.
I numeri rappresentabili usando 8 bit con il complemento a due sono:

$$2^8 = 256$$

In particolare il range è:

$$-128 \quad +127$$

Esempio:

Calcolare:

$$39 + (-91)$$

Per prima cosa convertiamo il valore assoluto dei due numeri in binario:

39 :

-128	64	32	16	8	4	2	1
0	0	1	0	0	1	1	1

$$00100111$$

91 :

-128	64	32	16	8	4	2	1
0	1	0	1	1	0	1	1

$$01011011$$

Per i numeri negativi bisogna poi svolgere l'ulteriore passaggio (complemento a due):

- Si invertono tutti i bit a partire da sinistra fino ad arrivare all'ultimo 1 del numero(escluso).

91:

$$01011011$$

-91:

$$10100101$$

A questo punto non rimane che sommare e abbiamo vinto:

$$00100111+$$

$$10100101 =$$

$$11001100$$

Questo numero corrisponde a -52, che è proprio quello che volevamo ottenere.

⚠ **Se si ottiene un numero negativo che però ha l'ultimo bit pari a 0, bisogna ulteriormente complementarlo a 2!**

Controllo scanf

Per evitare warnings ed errori vari:

```
    if (scanf("%s", giocScelto) == EOF) {
        fprintf(stderr, "Errore di lettura");
        return -1;
    }
```

Esempio Allocazione Dinamica

Allocazione dinamica memoria

Esempio di allocazione di un array di strutture a partire da una lista di strutture:

```
list elenco = emptyList();
int len = length(elenco);
Ordine* array = (Ordine*)malloc(len*sizeof(Ordine));
```

Inizializzare una struttura

```
element a = {0}
```

Inizializza tutto a 0.

Inizializzazione di vout per il Merge Sort

Il modo più corretto è allocare dinamicamente l'array:

```
void ordina(Giocatore* elenco, int dim) {
    Giocatore* vout = (Giocatore*) malloc(sizeof(Giocatore) * dim);
    if (vout == NULL) {
        fprintf(stderr, "Errore di allocazione");
        exit(1);
    }
    mergeSortGioc(elenco, 0, dim-1, vout);
    free(vout);
}
```

⚠ **Bisogna sempre passare a mergeSort dim-1 e non dim!**

⚠ **Ricordati sempre di deallocare `vout` !**

Particolarità fscanf

File di testo

Esempio lettura:

```c
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include "element.h"
#include "list.h"

int main() {
    // 1. Puntatore al file e apertura
    FILE *fp = fopen("dati.txt", "r");
    if (fp == NULL) {
        printf("Errore: impossibile aprire il file.\n");
        return -1;
    }

    list miaLista = emptyList(); // Inizializzo la lista vuota
    element e;                   // Variabile temporanea per la lettura

    // 2. Ciclo di lettura
    // fscanf restituisce il numero di campi letti correttamente
    while (fscanf(fp, "%s %d %d", e.nome, &e.info1, &e.info2) == 3) {
        // 3. Inserimento nella lista
        miaLista = cons(e, miaLista);
    }

    // 4. Chiusura file e visualizzazione
    fclose(fp);
    printf("Dati letti dal file:\n");
    showList(miaLista);

    // 5. Pulizia memoria (Fondamentale!)
    freeList(miaLista);
```

```
    return 0;
}
```

`fscanf` ignora sempre spazi bianchi o linee a capo.

Se ho delle stringhe di caratteri separate da spazi che devo leggere con `fscanf`, posso usare:

```
char buffer[256];

// Legge tutto finché non trova un \n (newline)
if (fscanf(file, " %[^\n]", buffer) == 1) {
    printf("Stringa con spazi: %s\n", buffer);
}
```

Se so il numero massimo di caratteri che questa stringa potrebbe contenere (ad es 255):

```
char buffer[256];

// Legge tutto finché non trova un \n (newline)
if (fscanf(file, " %255[^\n]", buffer) == 1) {
    printf("Stringa con spazi: %s\n", buffer);
}
```

⚠ **Occhio allo spazio prima di `%`! Serve affinché vengano ignorati eventuali spazi vuoti nel file.**

Se ho lo stesso problema con dei valori interi devo avere l'accortezza di mettere sempre nella struttura del valore di riferimento una variabile atta a memorizzare la dimensione logica dell'array.
Es:

```
typedef struct {
    char nome[21];
    char indicTe[5];
    int dim;
    int* quant;
} Acquisto;
```

Ora è possibile usare il sistema a due passate (che amo molto).

```
Acquisto leggiUnAcquisto(FILE* fp) {
    int valore;
    int i = 0;
    element a;
    //Controllo apertura
    if (fp == NULL) {
        fprintf(stderr, "Errore di apertura");
        strcpy(a.nome, "NA");
        strcpy(a.indicTe, "NA");
        return a;
    }
    //Prima passata
    if (fscanf(fp, "%s %s", a.nome, a.indicTe) != 2) {
        fprintf(stderr, "Errore di lettura");
        strcpy(a.nome, "NA");
        strcpy(a.indicTe, "NA");
        return a;
    }
    long segnalibro = ftell(fp);
    a.dim = 0;
    while (a.dim < 100 && fscanf(fp, "%d", &valore) == 1) {
        a.dim++;
    }
    //Individuata la lunghezza dell'array lo allochiamo e toriamo
indietro nel file
    a.quant = (int*)malloc(sizeof(int) * a.dim);
    if (a.quant == NULL) {
        fprintf(stderr, "Errore di allocazione");
        strcpy(a.nome, "NA");
        strcpy(a.indicTe, "NA");
        return a;
    }
    fseek(fp, segnalibro, SEEK_SET);
    //Seconda passata
    while (i < a.dim) {
        if (fscanf(fp, "%d", &a.quant[i]) != 1) {
```

```
            fprintf(stderr, "Errore di lettura");
            strcpy(a.nome, "NA");
            strcpy(a.indicTe, "NA");
            return a;
        }
        i++;
    }
    return a;
}
```

Se invece devo leggere dei dati separati non da spazi ma da separatori specifici posso fare così (la sintassi di scanf è molto simile a quella di fscanf):

```
int giorno, mese, anno;
// Legge una data nel formato gg-mm-aaaa
scanf("%d-%d-%d", &giorno, &mese, &anno);
```

Se devo fare lo stesso per stringhe che contengono spazi posso fare così:

```
char descrizione[100];
int codice;
// Legge tutto fino al punto e virgola, poi legge un intero
scanf("%[^;];%d", descrizione, &codice);
```

Se invece ho i singoli campi che ogni volta vanno a capo, devo fare così:
Per i campi fissi siamo chill, per le stringhe invece bisogna usare come `\n` come separatore.

```
int codice;
char descr[1024];
char tipo[4];

scanf("%d", &codice);           // Legge l'intero, ma lascia il '\n'
nel buffer
scanf(" %[^\n]", descr);        // Lo spazio iniziale salta il '\n' e
legge la riga
```

```
scanf("%s", tipo);              // Legge la stringa successiva (senza spazi)
```

Printf

Uso consapevole di `printf` :

> Il primo consiglio consiste nell'inserire sempre `\n` alla fine di un una stampa.

```
printf("Hello World!\n");
```

> Abbi in mente la sintassi di printf

```
%[flag][larghezza][.precisione][lunghezza]tipo$$
```

Flag:

Flag	Descrizione	Esempio
`-`	**Allineamento a sinistra** (default è a destra).	`%-10s`
`+`	Fornisce sempre il **segno** (`+` o `-`) per i numeri.	`%+d`
`0`	Riempie con **zeri iniziali** anziché spazi.	`%05d` (stampa `00123`)
`#`	Usa una forma alternativa (es. `0x` per esadecimale).	`%#x`

Larghezza:

Specifica la **larghezza minima** del campo di output. Se il valore stampato è più corto, viene riempito con spazi (o zeri, se è presente il flag `0`).

Precisione :

La precisione è preceduta da un punto (`.`) e ha diversi significati a seconda del tipo:

Tipo	Descrizione	Esempio
Float	Numero di cifre dopo la **virgola decimale**.	`printf("%.2f", 3.14159);` (stampa `3.14`)
Interi	Numero minimo di cifre che devono apparire.	`printf("%.5d", 123);` (stampa `00123`)
Stringhe	**Numero massimo** di caratteri da stampare.	`printf("%.3s", "Ciao");` (stampa `Cia`)

Modificatori di lunghezza:

Modifica la dimensione del tipo di argomento (es. `long int` anziché `int`).

Modificatore	Descrizione	Specificatore di Formato	Argomento Atteso	Esempio d'Uso
`l`	Per `long int` o `double` (`long double` richiede `%Lf`).	`%ld`, `%lu`, `%lf`	`long int`, `unsigned long int`, `double`	`printf("%ld", grande_int);`
`ll`	Per `long long int` o `unsigned long long int`.	`%lld`, `%llu`	`long long int`, `unsigned long long int`	`printf("%lld", molto_grande);`
`h`	Per `short int` o `unsigned short int`.	`%hd`, `%hu`	`short int`, `unsigned short int`	`printf("%hd", corto);`
`z`	Per il tipo `size_t` (usato per dimensioni e indici).	`%zu`	`size_t`	`printf("%zu", dimensione);`
`L`	Per il tipo `long double` (solo con `%f`, `%e`, `%g`).	`%Lf`, `%Le`, `%Lg`	`long double`	`printf("%Lf", numero_lungo);`

Specchietto retrovisore

```
void statistiche_raggruppate(Element* vet, int dim) {
    //Se l'array è vuoto
    if (dim <= 0 || vet == NULL) {
        return;
    }

    //Ordino l'array
    ordina(vet, dim);

    int contatore = 1;
    float somma = 0;

    // Parto da 1 (il secondo) e guardo indietro a [i-1]
    for (int i = 1; i < dim; i++) {

        // Confronto l'elemento corrente con il precedente
        if (strcmp(vet[i].nome, vet[i - 1].nome) == 0) {
            //Sono uguali
            contatore++;
            somma += vet[i].importo;
        }
        else {
            //Sono diversi, dunque cambio gruppo e stampo quello
precedente (quello di i-1)
            printf("L'autore %s ha: %d elementi, somma: %.2f\n", vet[i
- 1].nome, contatore, somma);

            //Resetto per il nuovo gruppo
            contatore = 1;
            somma = vet[i].importo;
        }
    }

    // Stampo i dati dell'ultimo elemento dell'array (dim-1).
    printf("L'autore %s ha: %d elementi\n", vet[dim - 1].nome,
```

```
contatore);

    return;
}
```

Stampa ricorsiva

Stampa la lista "al dritto":

```
void stampaLista(list lista) {
    if (empty(lista)) {
        printf("");
    }
    else {
        printElement(head(lista));
        stampaLista(tail(lista));
    }
}
```

Stampa la lista "alla rovescio":

```
void stampaLista(list lista) {
    if (empty(lista)) {
        printf("");
    }
    else {
        stampaLista(tail(lista));
        printElement(head(lista));
    }
}
```

String.h

String.h

1. `strcmp` (Confronto)

È la più importante. Serve per vedere se due stringhe sono uguali o quale viene prima in ordine alfabetico.

- **Sintassi:** `strcmp(s1, s2)`
- **Ritorna:** `0` se sono uguali, `< 0` se `s1` precede `s2`, `> 0` se `s1` segue `s2`.

```
if (strcmp(e.nome, "Mario") == 0) {
    printf("Sono uguali!");
}
```

2. `strcpy` (Copia)

In C non puoi fare `stringa1 = stringa2`. Devi "copiare" il contenuto da una parte all'altra.

- **Sintassi:** `strcpy(destinazione, sorgente)`

```
char nome[50];
strcpy(nome, "Luigi"); // Copia "Luigi" dentro l'array nome
```

3. `strlen` (Lunghezza)

Ti dice quanti caratteri ci sono nella stringa (escluso il carattere terminatore `\0`).

- **Sintassi:** `strlen(s1)`

```
char parola[] = "Ciao";
int len = strlen(parola); // len sarà 4
```

4. `strtok` (Scomposizione / Tokenizzazione)

Utilissima se hai una stringa complessa (es: `"Rossi;Mario;1990"`) e vuoi dividerla usando un separatore (il "token").

```
char riga[] = "Rossi;Mario;1990";
char *cognome = strtok(riga, ";");
// Prende "Rossi"
```

```
char *nome = strtok(NULL, ";");
//Prende "Mario" (passi NULL per continuare sulla stessa riga)
```

4. `strstr` (Scomposizione / Tokenizzazione)

```
char *strstr(const char *stringa_grande, const char
*parola_da_cercare);
```

Restituisce un puntatore alla prima lettera se trova la corrispondenza, NULL se non la trova.

Lettura Standard

Lettura liste

Ricorsivo:

```
list leggi_ricorsiva(FILE* f) {
    element e;

    // 1. Tentativo di lettura del prossimo elemento
    // (Assicurati che readElement restituisca il numero di campi corretti, es. 5)
    if (readElement(f, &e) != 5) {
        return emptyList(); // CASO BASE: Fine del file o errore di lettura
    }

    // 2. CHIAMATA RICORSIVA:
    // Legge il resto del file prima di montare la lista
    list restoDellaLista = leggi_ricorsiva(f);

    // 3. COSTRUZIONE:
    // Aggancia l'elemento corrente in testa al "resto" già letto
    return cons(e, restoDellaLista);
}
```

Iterativo:

```
list leggi_lista(char* fileName) {
    FILE* f = fopen(fileName, "r");
    if (f == NULL){
        fprintf(stderr, "Errore di allocazione\n");
        return emptyList();
    }

    //Variabile d'appoggio
    element e;

    list l = emptyList();
```

```
    //Lettura
    while (readElement(f, &e) == N) {
        l = cons(e, l);
    }

    fclose(f);

    //Ritorno la lista invertita (ordine del file)
    return reverse(l);
}
```

Lettura Array

Se l'array corrisponde al tipo element:

```
TIPO_ELEMENTO* leggi_array(char* fileName, int* dim) {
    FILE* f = fopen(fileName, "r");
    if (f == NULL){
        fprintf(stderr, "Errore di apertura\n");
        return NULL;
    }

    int conto = 0;
    int tmp = 0;
    //Prima passata di conto
    // Uso l'asterisco %* per scartare i dati che non mi servono ora
(ne tengo uno per il corretto funzionamento di fscanf)
    while (fscanf(f, "%d, %*FORMATO_SCARTO", &tmp) == 1) {
        conto++;
    }

    if (conto == 0) {
        fclose(f);
        return NULL;
    }

    //Allocazione array
    TIPO_ELEMENTO* array = (TIPO_ELEMENTO*)malloc(conto *
```

```
sizeof(TIPO_ELEMENTO));
    if (array == NULL) {
        fclose(f);
        fprintf(stderr, "Errore di allocazione\n");
        return NULL;
    }

    //Seconda passata di assegnamento
    rewind(f);
    for (int i = 0; i < conto; i++) {
        readElement(f, &array[i]);
    }
    fclose(f);
    //Assegnamento dimensione array
    *dim = conto;

    return array;
}
```

Altrimenti:

```
TIPO_SPECIFICO* leggi_array(char* fileName, int* dim) {
    FILE* f = fopen(fileName, "r");
    if (f == NULL){
        fprintf(stderr, "Errore di apertura\n");
        return NULL;
    }

    int conto = 0;
    int tmp = 0;
    //Prima passata di conto
    // Uso l'asterisco %* per scartare i dati che non mi servono ora
(ne tengo uno per il corretto funzionamento di fscanf)
    while (fscanf(f, "%d, %*FORMATO_SCARTO", &tmp) == 1) {
        conto++;
    }

    if (conto == 0) {
        fclose(f);
```

```
        return NULL;
    }

    //Allocazione array
    TIPO_SPECIFICO* array = (TIPO_SPECIFICO*)malloc(conto *
sizeof(TIPO_SPECIFICO));
    if (array == NULL) {
        fclose(f);
        fprintf(stderr, "Errore di allocazione\n");
        return NULL;
    }

    //Seconda passata di assegnamento
    rewind(f);
    for (int i = 0; i < conto; i++) {
        readArrayRecord(f, &array[i]);
    }
    fclose(f);
    //Assegnamento dimensione array
    *dim = conto;

    return array;
}
```

Lettura Master-Detail (con dim)

Struttura struct

Se il mio `detail` deve contenere più informazioni esso deve avere una struttura apposita.
Es:

```
typedef struct {
    int id;
    int quantita;
} TIPO_DETAIL; // o "Articolo", "SottoElemento", ecc.
```

```
typedef struct {
    char codice_ordine[20]; // Dati generali del Master
    int numeroCampiVaraibili;          // <--- CHIAVE 1: Dimensione logica dello specifico Detail
    TIPO_DETAIL* dettagli; // <--- CHIAVE 2: Puntatore per la malloc dei dettagli
} TIPO_MASTER; // Questo sarà il tuo "element"
```

Altrimenti (`detail` per esempio costituito da soli interi) basta una sola struttura:

```
typedef struct {
    char codice_gruppo[20];
    int n_interi;     // Quanti numeri seguono
    int* lista_id;    // Puntatore a intero (non a una struct)
} element;
```

Implementazione

```
TIPO_MASTER* leggi_master_detail(char* fileName, int* dim) {
    FILE* f = fopen(fileName, "r");
    if (f == NULL) {
        fprintf(stderr, "Errore di apertura");
        return NULL;
    }
    //Prima passata di conto
```

```
    int contoCampiFissi = 0;
    int numeroCampiVariabili = 0;
    //Scartiamo i campi fissi e leggiamo il numero di campi variabili
    while (fscanf(f, "FORMATO_CAMPI_FISSI %*d %d",
&numeroCampiVariabili) == 1) {
        // Salto i campi variabili
        for (int i = 0; i < numeroCampiVariabili; i++) {
            int tmp = 0;
            if (fscanf(f, " FORMATO_CAMPI_VARIABILI %* %d", &tmp) != 1)
{
                fprintf(stderr, "Errore di lettura");
                fclose(f);
                return NULL;
            }
        }
        contoCampiFissi++;
    }
    //Allocazione Array Campi Fissi
    TIPO_MASTER* arrayM = (TIPO_MASTER*)malloc(contoCampiFissi,
sizeof(TIPO_MASTER));
    if (arrayM == NULL) {
        fprintf(stderr, "Errore di allocazione");
        fclose(f);
        return NULL;
    }
    rewind(f);
    //Seconda passata di assegnamento
    for (int i = 0; i < contoCampiFissi; i++) {
        if (readElement(f, &arrayM[i]) <= 0) {
            fprintf(stderr, "Errore lettura elemento %d\n", i);
            //In questo caso devo anche liberare la memoria occuapata
dagli array già allocati
            free(arrayM);
            fclose(f);
            return NULL;
        }
    }
    fclose(f);
```

```
    *dim = contoCampiFissi;
    return arrayM;
}
```

Funzione di lettura:

```
int readArrayRecord(FILE* f, record* v) {
    //Legge solo un elemento dell'array
    //Se la lettura dei campi fissi fallisce
    if (fscanf(f, "%s %d", v->info1, &v->info2) != 2) {
        return 0; // Fine file o errore
    }
    int nCampiVariabili = v->n_dettagli
        //Allocazione array campi varaibili
        v->dettagli = (TIPO_DETTAGLI*)malloc(nCampiVariabili *
sizeof(TIPO_DETAIL));
    if (v->dettagli == NULL) {
        fprintf(stderr, "Errore di allocazione");
        return 0;
    }

    //Leggo le coppie (in questo caso separate da #)
    for (int j = 0; j < nCampiVariabili; j++) {
        if (fscanf(f, " %d#%d", &v->dettagli[j].val1, &v-
>dettagli[j].val2) != 2) {
            return 0;
        }
    }

    //Se la lettura è andata a buon fine
    return 1;
}
```

Funzione di stampa:

```
void printArrayRecord(record* v, int dim, char* etichetta) {
    printf("\n%s:\n", etichetta);
    if (v == NULL || dim <= 0) {
        printf("Nessun prodotto in elenco.\n");
```

```
        return;
    }

    for (int i = 0; i < dim; i++) {
        printf("[%02d] Nome: %-s | Info1: %-5d | Info2: %-5d\n", i + 1,
v[i].nome, v[i].info1, v[i].info2);
        if (v[i].dettagli != NULL) {
            for (int j = 0; j < v[i].n_dettagli; j++) {
                // Stampa dei campi variabili
                printf(" | Info1 variabile: %-5d | Info2 variabile:
%-5d\n", v[i].dettagli[j].info1, v[i].dettagli[j].info2);
            }
        }
    }
}
```

Lettura Master-Detail (detail statico)

In questo caso le strutture saranno così:

```
#define MAX_DETAIL 20

typedef struct {
    int id_prodotto;
    float prezzo;
} Detail;

typedef struct {
    int id_ordine;
    int n_detail;
    //Array Statico
    Detail lista[MAX_DETAIL];
} Master;
```

Funzioni di base:

⚠ **Occhio ad implementare sempre la macro `MAX_DETAIL`**

```
int readArrayRecord(FILE* f, record* v) {
    //Se non legge i campi fissi terminiamo
    if (fscanf(f, "%d %d", &v->id_ordine, &v->n_detail) != 2) {
        return 0;
    }

    //Lettura campi variabili
    for (int j = 0; j < v->n_detail; j++) {
        if (j < MAX_DETAIL) {
            if (fscanf(f, "%d %f", &v->lista[j].id_prodotto, &v-
>lista[j].prezzo) != 2) {
                //Errore di lettura
                return 0;
            }
        }
        else {
```

```
            //Se i campi variabili superano il limite li leggiamo per
avanzare nel file ma scaritamo i loro valori
            int tmpId;
            float tmpPrezzo;
            if (fscanf(f, "%d %f", &tmpId, &tmpPrezzo) != 2) {
                //Errore di lettura
                return 0;
            }
        }
    }

    //Lettura effettuata con successo
    return 1;
}
```

```
void printArrayRecord(record* v, int dim, char* etichetta) {
    printf("\n%s:\n", etichetta);
    if (v == NULL || dim <= 0) {
        printf("Nessun prodotto in elenco.\n");
        return;
    }

    for (int i = 0; i < dim; i++) {
        printf("[%02d] Nome: %-s | Info1: %-5d | Info2: %-5d\n", i + 1,
v[i].nome, v[i].info1, v[i].info2);
            for (int j = 0; j < v[i].n_dettagli; j++) {
                // Stampa dei campi variabili solo se non superno il
limite
            if(j<MAX_DETAIL){
                    printf(" | Info1 variabile: %-5d | Info2 variabile:
%-5.2f\n", v[i].lista[j].Id_prodotto, v[i].lista[j].prezzo);
            }
        }
    }
}
```

Implementazione (lettura di tutto l'array):
Se non conosco le dimensioni dei campi variabili uso questa funzione:

```
TIPO_MASTER* leggi_master_detail(char* fileName, int* dim) {
    FILE* f = fopen(fileName, "r");
    if (f == NULL) {
        fprintf(stderr, "Errore di apertura");
        return NULL;
    }

    //Prima passata di conto
    int contoCampiFissi = 0;
    // Leggiamo i campi fissi e il numero di campi variabili
    int stop = FALSE;
    int dummy = 0;
    while (fscanf(f, "FORMATO_CAMPI_FISSI %* %d", &dummy) == 1) {
        // Salto i campi variabili
        int tmp = 0;
        for (int i = 0; stop == FALSE && i < MAX_DETAIL; i++) {
            if (fscanf(f, " FORMATO_CAMPI_VARIABILI %* %d", &tmp) != 1)
{
                stop = TRUE;
            }
        }
        contoCampiFissi++;
    }

    //Allocazione Array Campi Fissi
    TIPO_MASTER* arrayM = (TIPO_MASTER*)malloc(contoCampiFissi *
sizeof(TIPO_MASTER));
    if (arrayM == NULL) {
        fprintf(stderr, "Errore di allocazione");
        fclose(f);
        return NULL;
    }
    rewind(f);
    //Seconda passata di assegnamento
    for (int i = 0; i < contoCampiFissi; i++) {
```

```
        if (readElement(f, &arrayM[i]) <= 0) {
            fprintf(stderr, "Errore lettura elemento %d\n", i);
            free(arrayM);
            fclose(f);
            return NULL;
        }
    }

    fclose(f);
    *dim = contoCampiFissi;
    return arrayM;
}
```

Se invece un parametro mi dice la lunghezza dei campi variabili:

```
TIPO_MASTER* leggi_master_detail(char* fileName, int* dim) {
    FILE* f = fopen(fileName, "r");
    if (f == NULL) {
        fprintf(stderr, "Errore di apertura");
        return NULL;
    }
    //Prima passata di conto
    int contoCampiFissi = 0;
    int numeroCampiVariabili = 0;
    //Scartiamo i campi fissi e leggiamo il numero di campi variabili
    while (fscanf(f, "FORMATO_CAMPI_FISSI %*d %d",
&numeroCampiVariabili) == 1) {
        // Salto i campi variabili
        for (int i = 0; i < numeroCampiVariabili; i++) {
            int tmp = 0;
            if (fscanf(f, " FORMATO_CAMPI_VARIABILI %* %d", &tmp) != 1)
{
                fprintf(stderr, "Errore di lettura");
                fclose(f);
                return NULL;
            }
        }
        contoCampiFissi++;
    }
```

```
    //Allocazione Array Campi Fissi
    TIPO_MASTER* arrayM = (TIPO_MASTER*)malloc(contoCampiFissi,
sizeof(TIPO_MASTER));
    if (arrayM == NULL) {
        fprintf(stderr, "Errore di allocazione");
        fclose(f);
        return NULL;
    }
    rewind(f);
    //Seconda passata di assegnamento
    for (int i = 0; i < contoCampiFissi; i++) {
        if (readElement(f, &arrayM[i]) <= 0) {
            fprintf(stderr, "Errore lettura elemento %d\n", i);
            //In questo caso devo anche liberare la memoria occuapata
dagli array già allocati
            free(arrayM);
            fclose(f);
            return NULL;
        }
    }
    fclose(f);
    *dim = contoCampiFissi;
    return arrayM;
}
```

Questa è la stessa funzione del caso in cui conosco le dimensioni dell'array dei campi variabili ma non conosco la loro lunghezza massima. La differenza sta solo nella `readElement`.

Lettura Master-Detail (senza dim)

Struttura struct

Se il mio `detail` deve contenere più informazioni esso deve avere una struttura apposita.
Es:

```
typedef struct {
    int id;
    int quantita;
} TIPO_DETAIL; // o "Articolo", "SottoElemento", ecc.
```

```
typedef struct {
    char codice_ordine[20]; // Dati generali del Master
    int numeroCampiVaraibili;          // <--- CHIAVE 1: Dimensione logica dello specifico Detail
    TIPO_DETAIL* dettagli; // <--- CHIAVE 2: Puntatore per la malloc dei dettagli
} TIPO_MASTER; // Questo sarà il tuo "element"
```

Altrimenti (`detail` per esempio costituito da soli interi) basta una sola struttura:

```
typedef struct {
    char codice_gruppo[20];
    int n_interi;    // Quanti numeri seguono
    int* lista_id;   // Puntatore a intero (non a una struct)
} element;
```

Implementazione

```
TIPO_MASTER* leggi_master_seza_dim(char* fileName, int* dimTotal) {
    FILE* f = fopen(fileName, "r");
    if (f == NULL){
        fprintf(stderr, "Errore di apertura\n");
         return NULL;
    }
```

```
    //Prima passata di conto
    int numeroCampiFissi = 0;
    //Trovo la dimensione dell' Array in base a quanti campi fissi
trovo
    while (fscanf(f, "FORMATO_CAMPI_FISSI", ...) == NUM_CAMPI_FISSI) {
        int temp;
        // Saltiamo i campi variabili finché la fscanf fallisce (cambio
tipo dato)
        while (fscanf(f, "FORMATO_SINGOLO_CAMPO_VARIABILE", &temp) ==
1);
        numeroCampiFissi++;
    }
    if (numeroCampiFissi == 0) {
    fprintf(stderr, "Il file è vuoto\n");
    fclose(f);
    return NULL;
    }
    //Allocazione array
    TIPO_MASTER* arrayM = malloc(numeroCampiFissi *
sizeof(TIPO_MASTER));
    if(arrayM == NULL){
        fprintf(stderr, "Errore di allocazione\n");
        fclose(f);
        return NULL;
    }
    rewind(f);

    //Seconda passata di assegnamento
    for (int i = 0; i < numeroCampiFissi; i++) {
        //Leggi i campi fissi
        if(fscanf(f, "FORMATO_CAMPI_FISSI", &arrayM[i].campo1, ...) !=
"NUMERO_FORMATO_CAMPI_FISSI"){
        fprintf(stderr, "Errore di lettura");
        fclose(f);
        liberatutto(arrayM, i);
        return NULL;
        }
```

```
        //Lista d'appoggio
        list listaTmp = emptylist();
        int contatore = 0;
        int valore_letto;

        //Leggo fino alla fine dell'elemento nel file
        while (fscanf(f, "FORMATO_SINGOLO_DETAIL", &valore_letto) == 1)
{
            element e;
            e.valore = valore_letto;
            listaTmp = cons(e, listaTmp);
            contatore++;
        }

        //Alloco l'array dei campi variabili della misura esatta
        arrayM[i].dim_dettagli = contatore;
        arrayM[i].dettagli = malloc(contatore * sizeof(int));
        if (arrayM[i].dettagli == NULL){
        fprintf(stderr, "Errore di allocazione\n");
        fclose(f);
        return NULL;
        }
        //Trasferisco i dati dalla lista all'array (la lista inserisce
in testa, quindi riempio l'array dalla fine)
        for (int j = contatore - 1; j >= 0; j--) {
            arrayM[i].dettagli[j] = head(listaTmp).valore;

            //Deallocazione
            list daLiberare = listaTmp;
            listaTmp = tail(listaTmp);
            free(daLiberare);
        }
    }
    fclose(f);
    *dimTotal = numeroCampiFissi;
    return arrayM;
}
```

⚠**Bisogna sempre deallocare usando la funzione seguente!**

Se il Master si presente come lista:

```
void libera_lista_con_array(Lista l) {
    while (l != NULL) {
        Lista temp = l;
        //Sposto il puntatore al nodo successivo
        l = l->next;

        //Libero i sotto-array
        if (temp->value.dettagli != NULL) {
            free(temp->value.dettagli);
        }

        //Libero il nodo della lista
        free(temp);
    }
}
```